

Title: Polling versus ereignisgesteuerter Betrieb (Interruptbetrieb)

Version: V3.4

Date: 26.06.2025

Document History

Version / Date	Author(s) email address	Changes and other notes
V 0.0 / 25.11.2023	gerhard.huessner@lehrbeauftragte.fhws.de	Aus V 2.9 / 11.5.2023 (032le) Poll. Vs. Interrupt entstanden
V 0.2 / 15.12.2023	gerhard.huessner@lehrbeauftragte.fhws.de	Programme getestet und eingefügt
V 2.0 / 10.3.2024	ludwig.eckert@thws.de	Dokument überarbeitet, Teilversuch 4 (Zusatzaufgabe) hinzugefügt
V 2.1 / 18.3.2024	ludwig.eckert@thws.de	Fehlerkorrekturen; Hinweis auf Messung der Stromaufnahme ergänzt.
V 2.2 / 25.3.2024	ludwig.eckert@thws.de	Bildnummerierung eingefügt
V 3.0 / 1.5.2024	ludwig.eckert@thws.de	Tabelle Messung der Stromaufnahme erweitert
V 3.1 / 6.6.2024	ludwig.eckert@thws.de	Interruptlogik Kap1.4 und Low Power Kap. 1.5 eingefügt
V 3.2 / 18.12.2024	ludwig.eckert@thws.de	Endlosschleife while(1; in Programmlisting eingefügt.
V 3.3 / 30.4.2025	ludwig.eckert@thws.de	LPM3 nach LPM2 in Kap. 5 geändert.
V 3.4 / 26.6.2025	ludwig.eckert@thws.de	Querverweise (Kap. 7.4) aktualisiert (Kap. 1.5) Testfragen 5.2.1 aktualisiert Interrupt-Wirkungslinie

Inhaltsverzeichnis

1	EINFÜHRUNG UND MOTIVATION	4	
1.1	Die Texas Instruments Mikrocontrollerfamilie MSP430F4xx.....	4	
1.1.1	Zentraleinheit	4	
1.1.2	Peripherieeinheiten	5	
1.1.3	Memory Map MSP430-44x - Mikrocontrollerfamilie.....	5	
1.2	Schaltpläne zum Mikrocontroller-Evaluation Board MSP430F449-STK2	6	
1.2.1	Interfacing für Input/Output-Peripheriemodule	7	
1.2.2	Pinbelegung am digitalen Input/Output Port 1	7	
1.2.3	LED-Ansteuerung über den digitalen Input/Output Port 1	8	
1.2.4	Tasterabfrage B5 und B6 am digitalen Input/Output Port 2 ...	8	
1.2.5	Konstantendefinition - Auszug aus dem Headerfile msp430x44x.h	9	
1.2.6	Sammel-Interrupt am digitalen Input/Output Port 1 u. 2	9	
1.3	Konfigurationsregister der digitalen PORT-Schnittstelle P1 u. P2	11	
1.3.1	Function Select Registers PnSEL -> Steuerregister P1SEL, P2SEL	12	12
1.3.2	Direction Registers PnDIR -> Steuerregister P1DIR, P2DIR	12	
1.3.3	Input Registers P1IN, P2IN -> Dateneingabe.....	12	
1.3.4	Output Registers P1OUT, P2OUT -> Datenausgabe.....	12	
1.3.5	Interrupt Enable P1IE, P2IE	12	
1.3.6	Interrupt-Edge Select P1IES	13	
1.3.7	Interrupt Flag Register P1IFG, P2IFG	13	
1.4	Interruptlogik des MSP430-449 STK2 Board.....	14	
1.5	Low Power Management des MSP430-449 STK2 Board.....	15	
2	TEILVERSUCH 0: INBETRIEBNAHME DES EMBEDDED SYSTEMS	17	
2.1	Ziele und Aufgabenbeschreibung	17	
2.2	Versuchsvorbereitung und Testfragen.....	17	
2.3	Testfragen.....	17	
2.4	Programmbeispiel	18	
2.5	Versuchsdurchführung und Anschluss des Boards	18	
2.6	Abzuliefernde Ergebnisse	20	
3	TEILVERSUCH 1: POLLING EINES TASTERS ZUR ANSTEUERUNG DER LED	21	21
3.1	Ziele und Aufgabenbeschreibung	21	
3.2	Versuchsvorbereitung und Testfragen.....	21	
3.3	Testfragen.....	21	
3.4	Programmbeispiel	21	
3.5	Versuchsdurchführung und Anschluss des Boards	22	
3.6	Abzuliefernde Ergebnisse	23	
4	TEILVERSUCH 2: INTERRUPT-BETRIEB.....	24	
4.1	Ziele und Aufgabenbeschreibung	24	
4.2	Versuchsvorbereitung und Testfragen.....	24	
4.3	Testfragen.....	24	
4.4	Programmbeispiel	24	
4.5	Versuchsdurchführung und Anschluss des Boards	25	
4.6	Abzuliefernde Ergebnisse	26	
5	TEILVERSUCH 3: ENERGIEMANAGEMENT DES MSP430F449 NUTZEN....	27	
5.1	Ziele und Aufgabenbeschreibung	27	
5.2	Versuchsvorbereitung und Testfragen.....	27	
5.2.1	Testfragen	28	
5.3	Programmbeispiel und Funktionsweise	28	
5.4	Versuchsdurchführung und Anschluss des Boards	30	
5.5	Abzuliefernde Ergebnisse	31	

6	TEILVERSUCH 4: TRANSFERAUFGABE	32
6.1	Ziele und Aufgabenbeschreibung	32
6.2	Versuchsdurchführung und Anschluss des Boards	32
6.3	Abzuliefernde Ergebnisse	33
6.4	Programmbeispiel und Lösung	33
6.5	Abzuliefernde Ergebnisse	34
7	ANHANG.....	35
7.1	Strommessung am MSP430-449 STK2 Board	35
7.2	Verwendete Hardware	36
7.3	Handhabung der Embedded Toolchain	36
7.3.1	Virtuelle Maschine	36
7.3.2	IAR Embedded Workbench - Starten eines neuen Projekts in der IDE	37
7.3.3	Kompilieren und Downloaden des Programmes in das Zielsystem	37
7.4	Feedbackformular zum Versuch	38

Abbildungsverzeichnis

Bild 1-1: MSP430x44x functional block diagrams	4
Bild 1-2: MSP430x44x Speicherorganisation (Memory Map)	5
Bild 1-3: Schaltplan zum Mikrocontroller-Evaluation Board MSP430F449-STK2	6
Bild 1-4: Register zur Ansteuerung der digitalen Input/Output-Peripheriemodule Port 1 u. 2	7
Bild 1-5: Pinbelegung Port 1 (Button B1 - B4)	7
Bild 1-6: LED-Interfacing (Portpin P1.3)	8
Bild 1-7: Interfacing Taster B5 (schwarz) -> Portpin P2.1	8
Bild 1-8: Interfacing Taster B6 (rot) -> Portpin P2.2	8
Bild 1-9: Konstantendefinition - Auszug aus dem Headerfile msp430x44x.h	9
Bild 1-10: Interruptlogik und Interrupt Aggregation (Sammelinterrupt) am I/O-Port des Texas Instruments Mikrocontrollers MSP 430F449	10
Bild 1-11: Interruptvektoren des MSP430x449	10
Bild 1-12: Die Control-, Daten- und Status-Register der digitalen Ein- und Ausgabeports (P1 u. P2)	11
Bild 1-13: Auszug aus dem Headerfile msp430x44x.h	11
Bild 7-3: Verfügbare Interrupt Vektoren der MSP430F44x-Familie	14
Bild 2-1: Anschluss des MSP430 Boards mit Spannungsversorgung über die JTAG-Schnittstelle	18
Bild 7-1: Vorher	35
Bild 7-2: Nachher	35
Tabelle 1: Stromaufnahme des MSP430-449 STK2 Boards	28

1 EINFÜHRUNG UND MOTIVATION

Dieser Versuch veranschaulicht am praktischen Beispiel einer Taster-Eingabe und LED-Ausgabe den Unterschied zwischen Polling und ereignisgesteuertem Betrieb. Im ereignisgesteuerten Betrieb wird zudem das Energiemanagement (Low Power Modus) des MSP430F449 genutzt.

Dem Versuch liegt ein Embedded System STK2 der Firma Olimex¹ zugrunde, welches den Mikrocontroller MSP430F449 von Texas Instruments verwendet. Zur Softwareentwicklung kommt die integrierte Entwicklungsumgebung Embedded Workbench von IAR Systems² zum Einsatz. Die Programmerstellung erfolgt in der Programmiersprache „C“.

Folgende Ziele werden mit diesem Versuch verfolgt:

- Kennenlernen der Entwicklungsumgebung IAR Embedded Workbench
- Kennenlernen des Interruptsystems des MSP430F449 zur Verwendung eines ereignisgesteuerten Betriebs
- Kennenlernen von Maßnahmen zur Energieeinsparung

Die Voraussetzung für die Bearbeitung dieses Versuches sind Grundkenntnisse in der Programmiersprache C sowie Kenntnisse über die verwendete Hardware, siehe MSP430_UserGuide_slau056l.pdf, Kap.17, Kap. 2.3, u.a.

1.1 Die Texas Instruments Mikrocontrollerfamilie MSP430F4xx

1.1.1 Zentraleinheit

Die Zentraleinheit (Central Processing Unit CPU) stellt den Rechnerkern dar, um den sich alles dreht. Sie realisiert die Ablaufsteuerung des Rechners selbst, die Durchführung arithmetisch-logischer Operationen, den Transfer von Daten und den Ablauf des Programms.

Bei der Familie MSP430 handelt es sich um eine 16-Bit-RISC-CPU mit verschiedenen universellen Registern, einem breiten Spektrum an Adressierarten, einem kleinen, aber flexiblen Befehlssatz, alles ausgelegt auf hohe Leistung bei besonders kleinem Stromverbrauch.

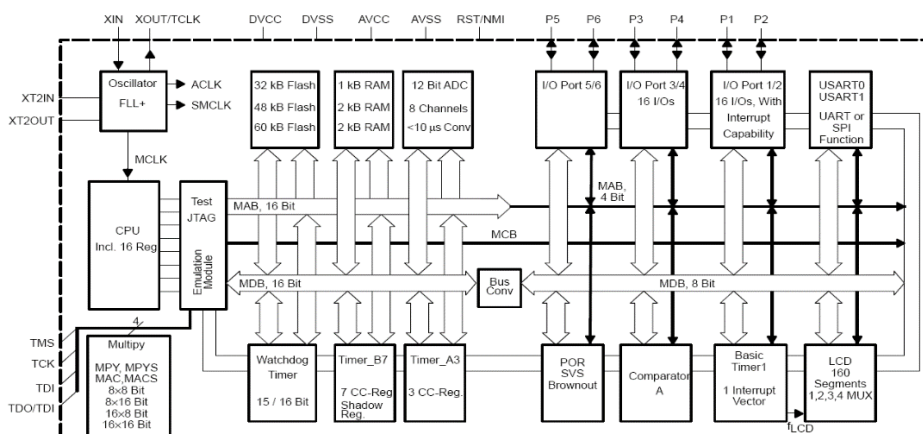


Bild 1-1: MSP430x44x functional block diagrams

¹ www.olimex.com

² www.iar.com

Zum Test des gesamten Systems und zur Programmierung ist die CPU von einer JTAG-Schnittstelle umgeben, die es erlaubt, auf den gesamten Mikrocontroller, einschließlich der Memory Bausteine, mit nur vier Leitungen effektiv zuzugreifen.

1.1.2 Peripherieeinheiten

Die Mikrocontrollerfamilie MSP430x4xx des Herstellers Texas Instruments zeichnet sich durch eine breite Palette intelligenter und von der CPU weitestgehend unabhängiger Peripherie aus, die die Realisierung komplexer Systeme mit nur geringer externer Beschaltung erlaubt.

Die Peripherie ist in Modulen realisiert, die mit der CPU durch den Speicheradressbus MAB, den Speicherdatenbus MDB und Unterbrechungsanforderungsleitungen (=Interruptleitungen) verbunden sind.

Die Befehlsausführung bei 16-Bit-Modulen arbeitet ohne Einschränkungen, z.B. Watchdog, ADC, TIMER A, Hardware-Multiplier etc.. Die Peripheriemodule arbeiten byteorientiert.

1.1.3 Memory Map MSP430-44x - Mikrocontrollerfamilie

		Funktion	Zugriff	Inhalt
0FFFFH	Unterbrechungsvektorentabelle	ROM	W/B	Vektoren
0FFE0H				
0FFDFH	Festwertspeicher	ROM	W/B	Programm, Tabellen, Konstanten
	nichts	keine		
0200H	Schreib-Lese-Speicher	RAM	W/B	Daten, Stapelspeicher, Programm
01FFH	16-Bit-Peripherie-Module	Timer, ADC,..	Wort	Peripherie-daten/steuerung
0100H				
0FFH	8-Bit-Peripherie-Module	I/O, LCD, Timer,..	Byte	Peripherie-daten/steuerung
010H				
0FH	Spezialregister	SFR	Byte	Systemdaten/-steuerung
0				

Bild 1-2: MSP430x44x Speicherorganisation (Memory Map)

1.2 Schaltpläne zum Mikrocontroller-Evaluation Board MSP430F449-STK2

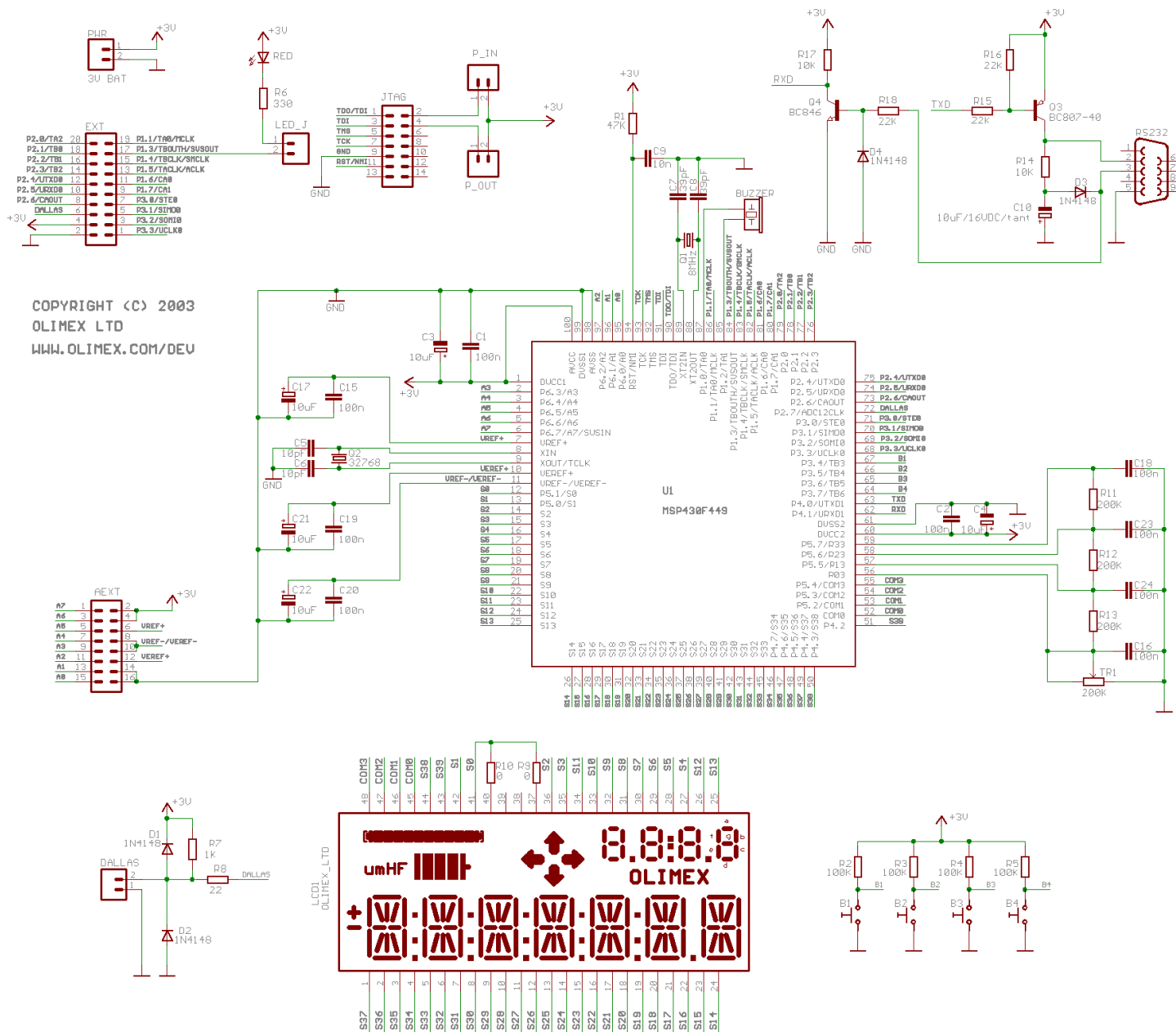
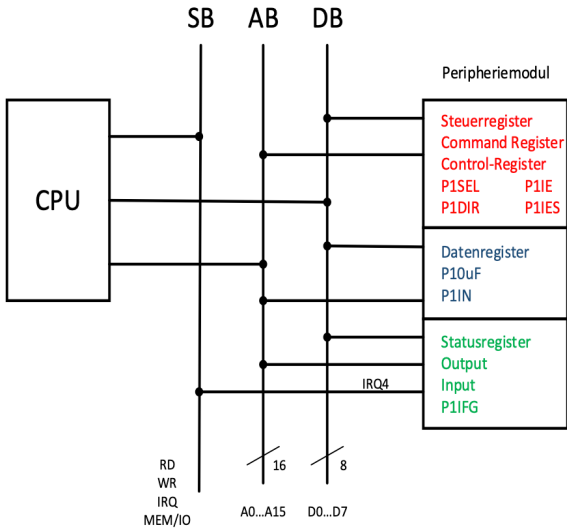


Bild 1-3: Schaltplan zum Mikrocontroller-Evaluation Board MSP430F449-STK2

1.2.1 Interfacing für Input/Output-Peripheriemodule



Steuerregister: dienen der Festlegung der Betriebsart (Command-Register, Control, CTL, Konfigurationsregister).

PxSEL – Pin Select

PxDIR – Pin Direction

PxIE – P1 Interrupt Enable

PxIES – P1 Interrupt Edge Select

Datenregister: dienen dem Austausch von Daten zwischen der CPU und dem Peripheriemodul.

```
char temp = P1IN;
```

```
P2OUT = temp;
```

Statusregister: geben Auskunft über den Zustand des Peripheriemoduls, z. B. Informieren über das Eintreffen neuer Datenbytes oder geben Auskunft über evtl. aufgetretene Übertragungsfehler oder Signalwechsel während der Übertragung.

PxIFG – P1 Interrupt Flag

Bild 1-4: Register zur Ansteuerung der digitalen Input/Output-Peripheriemodule Port 1 u. 2

1.2.2 Pinbelegung am digitalen Input/Output Port 1

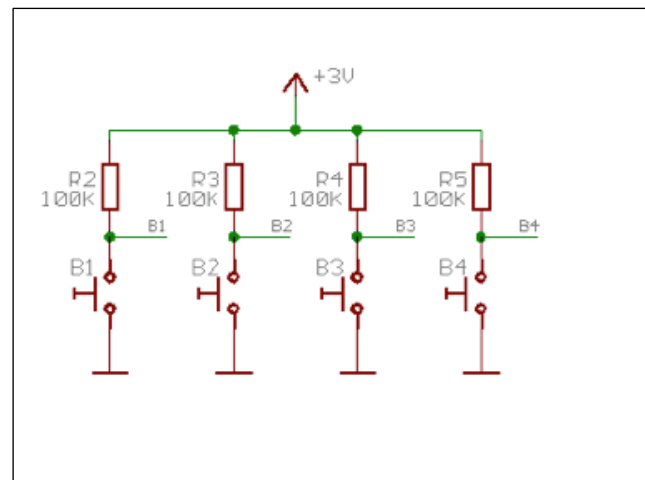
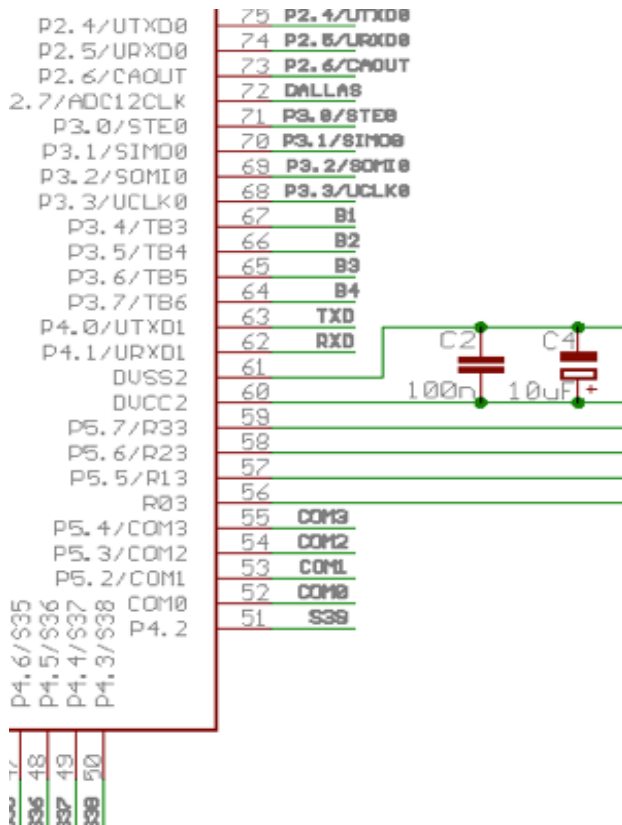


Bild 1-5: Pinbelegung Port 1 (Button B1 - B4)

1.2.3 LED-Ansteuerung über den digitalen Input/Output Port 1

Die LED ist über Portpin 1.3 des Texas Instruments Mikrocontrollers MSP430F449 angeschlossen.

Anmerkung:

Ein Low-Pegel an der Kathode der LED schaltet die LED ein, ein High-Pegel schaltet die LED aus.

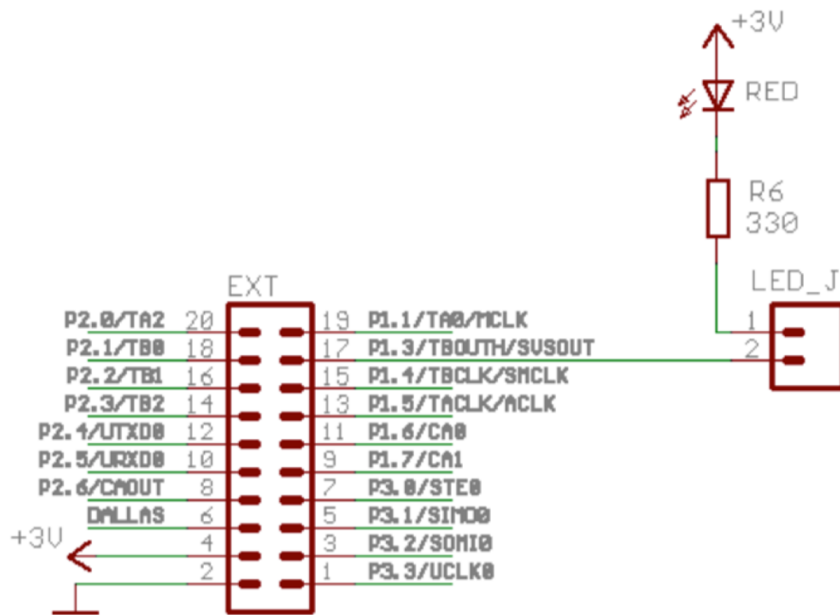


Bild 1-6: LED-Interfacing (Portpin P1.3)

1.2.4 Tasterabfrage B5 und B6 am digitalen Input/Output Port 2

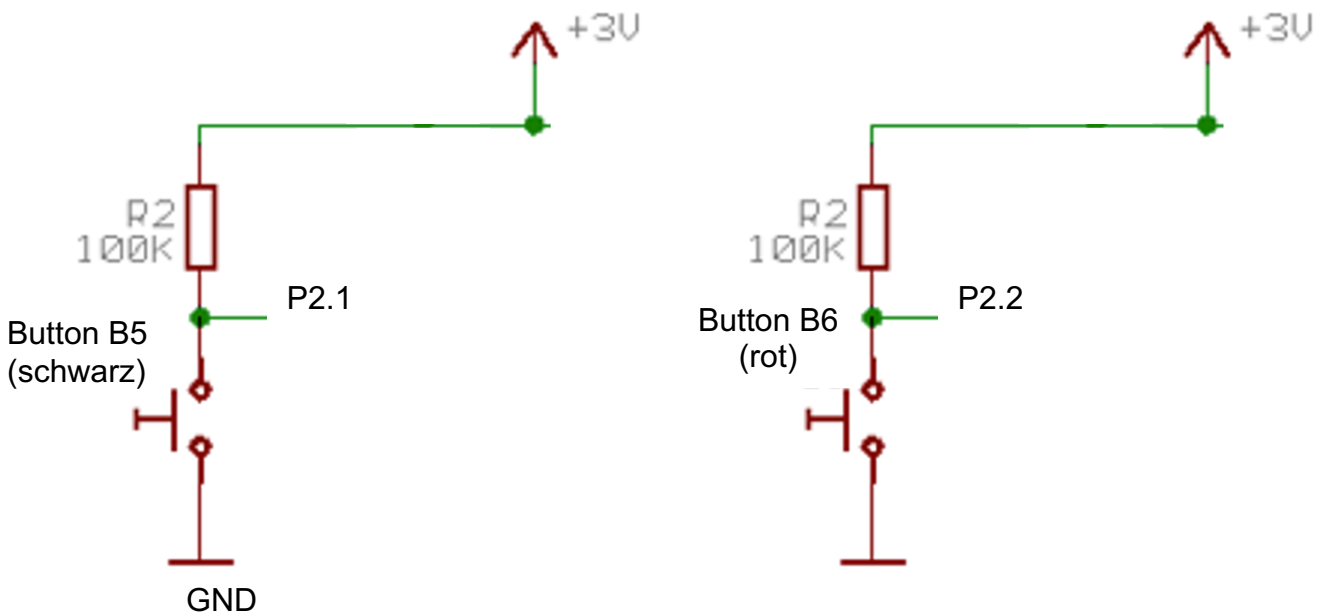


Bild 1-7: Interfacing Taster B5 (schwarz) -> Portpin P2.1

Bild 1-8: Interfacing Taster B6 (rot) -> Portpin P2.2

1.2.5 Konstantendefinition - Auszug aus dem Headerfile msp430x44x.h

```
/** STANDARD BITS ***/
```

```
#define BIT0    (0x0001)
#define BIT1    (0x0002)
#define BIT2    (0x0004)
#define BIT3    (0x0008)
#define BIT4    (0x0010)
#define BIT5    (0x0020)
#define BIT6    (0x0040)
#define BIT7    (0x0080)
#define BIT8    (0x0100)
#define BIT9    (0x0200)
#define BITA    (0x0400)
#define BITB    (0x0800)
#define BITC    (0x1000)
#define BITD    (0x2000)
#define BITE    (0x4000)
#define BITF    (0x8000)
```

Tabelle: Umwandlung Dezimal - Binär - Hexadezimal

Dezimal	Binär	Hexadezimal
	0b.....	0x..
0	0b00000000	0x00
1	0b00000001	0x01
2	0b00000010	0x02
3	0b00000011	0x03
4	0b00000100	0x04
5	0b00000101	0x05
6	0b00000110	0x06
7	0b00000111	0x07
8	0b00001000	0x08
9	0b00001001	0x09
10	0b00001010	0x0A
11	0b00001011	0x0B
12	0b00001100	0x0C
13	0b00001101	0x0D
14	0b00001110	0x0E
15	0b00001111	0x0F

Bild 1-9: Konstantendefinition - Auszug aus dem Headerfile msp430x44x.h

1.2.6 Sammel-Interrupt am digitalen Input/Output Port 1 u. 2

Die parallelen Ports 1 und 2 sind mit einer umfangreichen Interruptlogik ausgestattet, die vielfältig genutzt werden kann.

Prinzipiell sind alle Portpins der parallelen Ports 1 und 2 des MSP430 interruptfähig, unabhängig davon, ob sie als Ein- oder Ausgänge konfiguriert wurden. Port 1 und 2 verfügen über separate Interruptvektoren innerhalb der Interrupt Vektor Tabelle, siehe PORT1_VECTOR und PORT2_VECTOR.

Die Interruptvektoren PORT1_VECTOR und PORT2_VECTOR werden auch als Sammelinterrupts bezeichnet, da sie alle Interruptereignisse der jeweiligen Portpins sammeln.

Es ist also allein durch den ausgelösten Port-Interrupt nicht klar ist, an welchem Portpin ein Signalwechsel stattgefunden hat.

Zur genauen Identifizierung des Interruptereignisses muss das Interruptflagregister P1IFG bzw. P2IFG verwendet werden. Ein gesetztes Bit im P2IFG zeigt an, dass am korrespondierenden Portpin der im Flankenauswahlregister P1IES bzw. P2IES festgelegte Pegelwechsel erkannt wurde.

Kommt es also zum Interrupt, so muss innerhalb der Interrupt Service Routine zuerst mit Hilfe des Interruptflagregisters P1IFG bzw. P2IFG das auslösende Portpin detektiert werden.

Am Ende der Interrupt Service Routine muss das entsprechende Bit im Interruptflagregister P1IFG bzw. P2IFG gelöscht werden (auf 0 setzen), damit nicht derselbe Interrupt erneut und wiederholend ausgelöst wird.

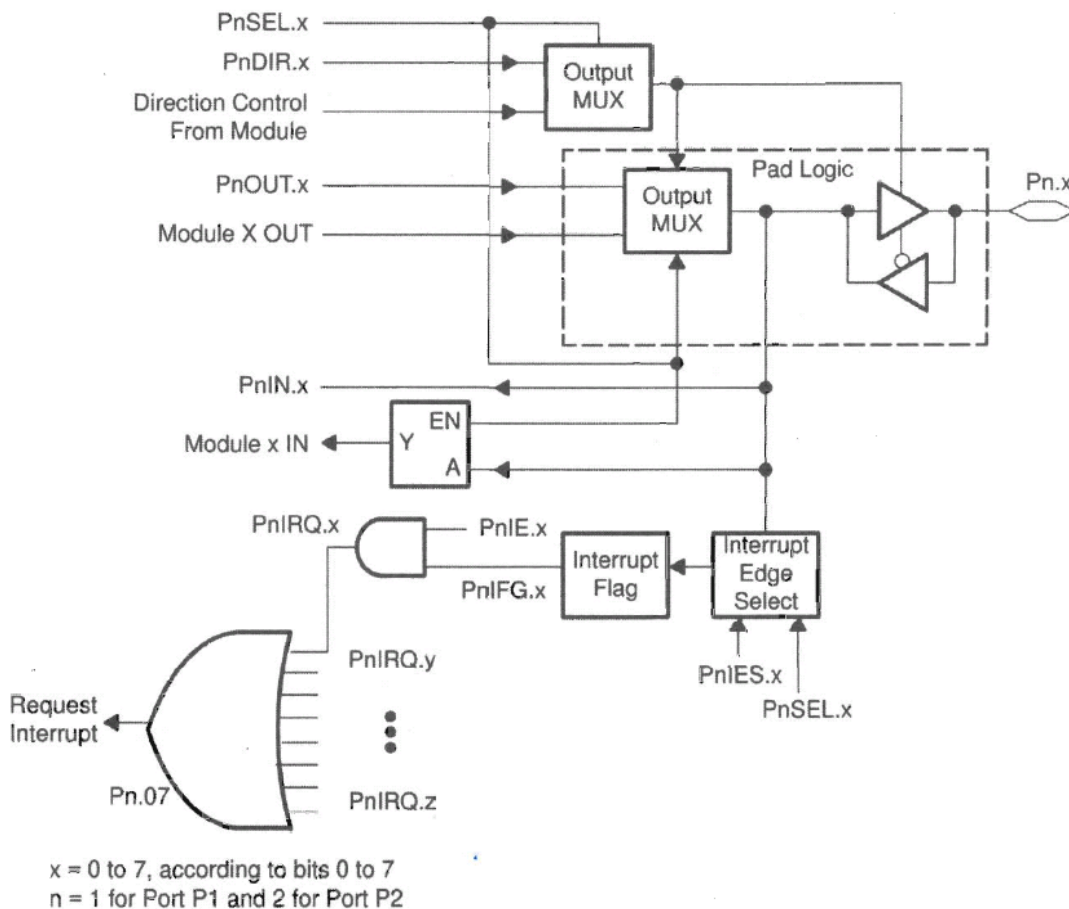


Bild 1-10: Interruptlogik und Interrupt Aggregation (Sammelinterrupt) am I/O-Port des Texas Instruments Mikrocontrollers MSP 430F449

```

/*****
* Interrupt Vectors (offset from 0xFFE0)
*****/
#define BASICTIMER_VECTOR    (0 * 2u) /* 0xFFE0 Basic Timer */
#define PORT2_VECTOR         (1 * 2u) /* 0xFFE2 Port 2 */
#define USART1TX_VECTOR      (2 * 2u) /* 0xFFE4 USART 1 Transmit */
#define USART1RX_VECTOR      (3 * 2u) /* 0xFFE6 USART 1 Receive */
#define PORT1_VECTOR         (4 * 2u) /* 0xFFE8 Port 1 */
#define TIMERA1_VECTOR        (5 * 2u) /* 0xFFEA Timer A CC1-2, TA */
#define TIMERA0_VECTOR        (6 * 2u) /* 0xFFEC Timer A CC0 */
#define ADC_VECTOR           (7 * 2u) /* 0xFFEE ADC */
#define USART0TX_VECTOR      (8 * 2u) /* 0xFFF0 USART 0 Transmit */
#define USART0RX_VECTOR      (9 * 2u) /* 0xFFF2 USART 0 Receive */
#define WDT_VECTOR           (10 * 2u) /* 0xFFF4 Watchdog Timer */
#define COMPARATORA_VECTOR    (11 * 2u) /* 0xFFF6 Comparator A */
#define TIMERB1_VECTOR        (12 * 2u) /* 0xFFF8 Timer B CC1-6, TB */
#define TIMERB0_VECTOR        (13 * 2u) /* 0xFFFA Timer B CC0 */
#define NMI_VECTOR           (14 * 2u) /* 0xFFFC Non-maskable */
#define RESET_VECTOR          (15 * 2u) /* 0xFFFE Reset [Highest Priority] */
#define UART1TX_VECTOR        USART1TX_VECTOR
#define UART1RX_VECTOR        USART1RX_VECTOR
#define UART0TX_VECTOR        USART0TX_VECTOR
#define UART0RX_VECTOR        USART0RX_VECTOR
/*****/

```

Bild 1-11: Interruptvektoren des MSP430x449

1.3 Konfigurationsregister der digitalen PORT-Schnittstelle P1 u. P2

Die Control-, Daten- und Status-Register der digitalen Ein- und Ausgabeports P1 u. P2

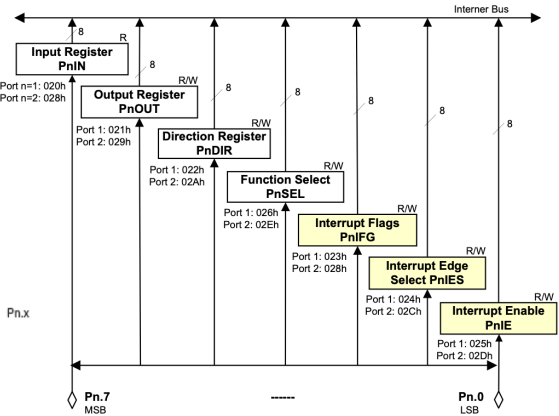
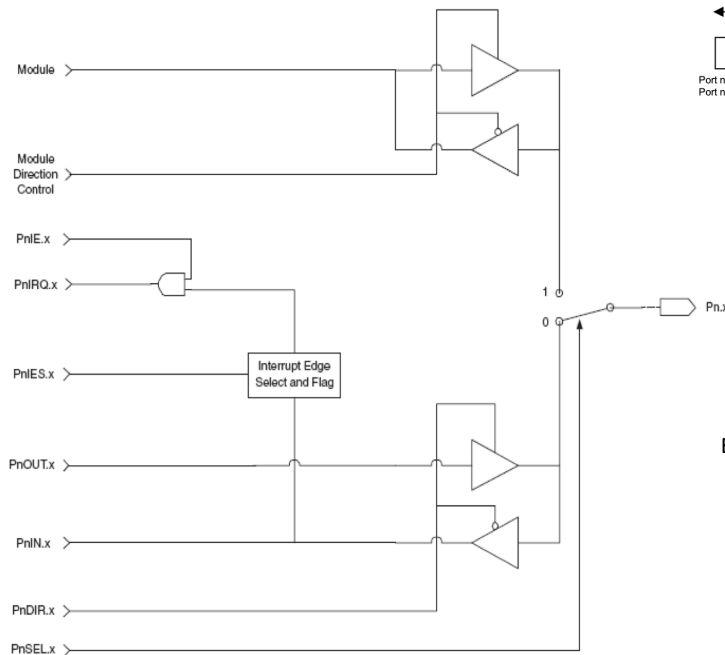


Bild: Digital I/O-Ports mit Interruptfähigkeit (Port 1 u. 2)

Steuerregister: P1SEL
P1DIR
P1IE
P1IES

Datenregister: P1OUT // read/write
P1IN // read only

Legende:
R – Read
R/W – Read / Write

Port	Register	Short Form	Address	Register Type	Initial State
P1	Input	P1IN	020h	Read only	–
	Output	P1OUT	021h	Read/write	Unchanged
	Direction	P1DIR	022h	Read/write	Reset with PUC
	Interrupt Flag	P1IFG	023h	Read/write	Reset with PUC
	Interrupt Edge Select	P1IES	024h	Read/write	Unchanged
	Interrupt Enable	P1IE	025h	Read/write	Reset with PUC
	Port Select	P1SEL	026h	Read/write	Reset with PUC
	Resistor Enable	P1REN	027h	Read/write	Reset with PUC

Bild 1-12: Die Control-, Daten- und Status-Register der digitalen Ein- und Ausgabeports (P1 u. P2)

```

/*****
* DIGITAL I/O Port1/2
*****/

#define P1IN          (0x0020) /* Port 1 Input */
#define P1OUT         (0x0021) /* Port 1 Output */
#define P1DIR         (0x0022) /* Port 1 Direction */
#define P1IFG         (0x0023) /* Port 1 Interrupt Flag */
#define P1IES         (0x0024) /* Port 1 Interrupt Edge Select */
#define P1IE          (0x0025) /* Port 1 Interrupt Enable */
#define P1SEL         (0x0026) /* Port 1 Selection */

#define P2IN          (0x0028) /* Port 2 Input */
#define P2OUT         (0x0029) /* Port 2 Output */
#define P2DIR         (0x002A) /* Port 2 Direction */
#define P2IFG         (0x002B) /* Port 2 Interrupt Flag */
#define P2IES         (0x002C) /* Port 2 Interrupt Edge Select */
#define P2IE          (0x002D) /* Port 2 Interrupt Enable */
#define P2SEL         (0x002E) /* Port 2 Selection */

```

Bild 1-13: Auszug aus dem Headerfile msp430x44x.h

1.3.1 Function Select Registers PnSEL -> Steuerregister P1SEL, P2SEL

Die P1- und P2-Portpins werden oft mit anderen Peripheriemodulen gemultiplext, um die Gesamtzahl der Pins bei MSP430-Devices zu reduzieren, s. a. MSP430 Datenblatt.

Die Steuerregister P1SEL und P2SEL werden verwendet, um die gewünschte Pin-Funktion, z. B. digitaler I/O-Port oder ein anderes Peripheriemodul, auszuwählen. Jedes Register enthält acht Bits, die jedem Pin des Hardwareports entsprechen. Die Funktion jedes Pins ist individuell wählbar.

Alle Bits in diesen Registern werden durch das Power Up Clear (PUC)-Signal zurückgesetzt. Die Bit-Definitionen sind wie folgt:

Bit = 0: Das Pin x von Port P1 / P2 arbeitet als digitales I/O

Bit = 1: Das Pin x von Port P1 / P2 ist einem anderen Peripheriemodul zugeordnet

1.3.2 Direction Registers PnDIR -> Steuerregister P1DIR, P2DIR

Die Richtungsregister enthalten acht unabhängige Bits, die die Richtung des I/O-Pins festlegen. Alle Bits werden durch das PUC-Signal zurückgesetzt. Wenn

Bit = 0: Der Portpin ist auf Eingangsrichtung geschaltet (3-State)

Bit = 1: Der Portpin ist in Ausgangsrichtung geschaltet

1.3.3 Input Registers P1IN, P2IN -> Dateneingabe

Beide Input-Register sind Nur-Lese-Register, die die Signale an den I/O-Pins wiedergeben.

1.3.4 Output Registers P1OUT, P2OUT -> Datenausgabe

Jedes Ausgangsregister zeigt die Informationen des Ausgangspuffers an. Der Ausgangspuffer kann von allen Befehlen, die auf ein Ziel schreiben, verändert werden.

Wird das Ausgangsregister zurückgelesen, so ist der Inhalt des Ausgangspuffers unabhängig von der Pin-Richtung. Ein Richtungswechsel verändert den Inhalt des Ausgangspuffers nicht.

1.3.5 Interrupt Enable P1IE, P2IE

Jedes Interrupt-Freigabe-Register (interrupt enable register) enthält Bits zur Freigabe des Interrupt-Flags für jeden E/A-Pin des Ports. Jedes der sechzehn Bits, die den Pins P1.0 bis P1.7 und P2.0 bis P2.7 entsprechen, befindet sich in den Registern P1IE und P2IE. Wenn

Bit = 0: Die Interrupt-Anforderung ist gesperrt

Bit = 1: Die Interruptanforderung ist aktiviert

Hinweis: Nur Port P1, Port P2 sind interruptfähig. Nur Übergänge, nicht statische Pegel, lösen Interrupts aus.

Ist bei der Ausführung des RETI-Befehls noch ein Interrupt-Flag gesetzt, z. B. bei einer Transition während der Interrupt-Service-Routine, kommt es nach Abschluss von RETI erneut zu einem Interrupt. Damit ist sichergestellt, dass jeder Übergang von der Software quittiert wird.

1.3.6 Interrupt-Edge Select P1IES

Dieses Lese-/Schreibregister wählt den Übergang aus, bei dem ein Interrupt ausgelöst werden soll.

$PnIES.x = 1$: Wenn es **gesetzt** ist, tritt ein **Interrupt bei einem High-Low-Übergang** an dem entsprechenden Pin auf.

$PnIES.x = 0$: Wenn es **gelöscht ist**, tritt ein **Interrupt bei einem Low-to-High-Übergang** an dem entsprechenden Pin auf.

1.3.7 Interrupt Flag Register P1IFG, P2IFG

Jedes $PnIFGx$ -Bit ist dem Interrupt-Flag für seinen entsprechenden I/O-Pin zugeordnet.

Das entsprechende $PnIFGx$ -Bit in diesem Lese-/Schreibregister wird automatisch gesetzt, wenn ein Interrupt erzeugt wird. D.h. das $PnIFGx$ -Bit wird gesetzt, wenn die ausgewählte Eingangssignalfanke (High->Low oder Low->High) an dem Pin auftritt.

Alle $PnIFGx$ -Interrupt-Flags lösen einen Interrupt aus, wenn ihr entsprechendes $PnIE$ -Bit und das Global Interrupt Enable (GIE)-Bit gesetzt sind. Dieses Register kann manuell beschrieben werden und erzeugt so per Software einen Interrupt, wenn ein High-Pegel geschrieben wird.

Jedes $PnIFG$ -Flag muss per Software zurückgesetzt werden. Wenn ein Interrupt auftritt, muss dieses Flag vor dem RETI-Befehl gelöscht werden, sonst ruft derselbe Interrupt die ISR ein zweites Mal auf.

Hinweise:

- Jedes $PnIFG$ -Flag kann auch per Software gesetzt werden, so dass ein softwareinitiiertes Interrupt erzeugt werden kann (Softwareinterrupt).
Bit = 0: Es steht kein Interrupt an
Bit = 1: Es steht ein Interrupt an
- Nur Übergänge, nicht statische Pegel, verursachen Interrupts.
- Wird ein $PnIFGx$ -Flag innerhalb einer Pn -Interrupt Servicerroutine gesetzt, erzeugt das gesetzte $PnIFGx$ -Flag einen weiteren Interrupt. Dadurch wird sichergestellt, dass jeder Übergang quittiert wird.

1.4 Interruptlogik des MSP430-449 STK2 Board

Das Verständnis der Interrupt Technik ist eine der wichtigsten Voraussetzungen bei der Handhabung von Mikrocontrollern.

Interrupt bedeutet die Unterbrechung des aktuell laufenden Programms, um schnellstmöglich auf ein plötzlich eingetretenes Ereignis reagieren zu können. Die Interrupttechnik entlastet die CPU von Überwachungsaufgaben (Polling), spart Zeit und in Verbindung mit dem Einsatz von Low Power Modi (LPM) Energie.

Ein Mikrocontroller ist prinzipiell in der Lage, auf Ereignisse unmittelbar zu reagieren. Solche Ereignisse können Pegelwechsel an Eingangs- oder Ausgangspins des Bausteins sein, hervorgerufen durch einen Tastendruck oder andere internen oder externen Ereignisse.

Bei Einsatz von Interrupts wird die CPU erst durch das Auftreten des Ereignisses selbst dazu veranlasst, auf dieses zu reagieren. Die CPU kann bei der Verwendung der Interrupttechnik folglich die meiste Zeit mit anderen als den Überwachungsaufgaben befasst sein oder sich in einem Strom sparenden Ruhezustand befinden.

Die Befehlsabfolge, die das Interrupt Ereignis bedient, wird Interrupt Service Routine oder kurz ISR genannt. Die ISR ist im Wesentlichen wie eine C-Funktion aufgebaut.

Für folgende Ereignisse können als ISR initialisiert werden:

```
#define BASICTIMER_VECTOR    (0 * 2) /* 0xFFE0 Basic Timer */
#define PORT2_VECTOR        (1 * 2) /* 0xFFE2 Port 2 */
#define UART1TX_VECTOR      (2 * 2) /* 0xFFE4 UART 1 Transmit */
#define UART1RX_VECTOR      (3 * 2) /* 0xFFE6 UART 1 Receive */
#define PORT1_VECTOR        (4 * 2) /* 0xFFE8 Port 1 */
#define TIMERA1_VECTOR       (5 * 2) /* 0xFFEA Timer A CC1-2, TA */
#define TIMERA0_VECTOR       (6 * 2) /* 0xFFEC Timer A CC0 */
#define ADC_VECTOR          (7 * 2) /* 0xFFEE ADC */
#define UART0TX_VECTOR      (8 * 2) /* 0xFFF0 UART 0 Transmit */
#define UART0RX_VECTOR      (9 * 2) /* 0xFFF2 UART 0 Receive */
#define WDT_VECTOR          (10 * 2) /* 0xFFF4 Watchdog Timer */
#define COMPARATORA_VECTOR  (11 * 2) /* 0xFFF6 Comparator A */
#define TIMERB1_VECTOR      (12 * 2) /* 0xFFF8 Timer B 1-7 */
#define TIMERB0_VECTOR      (13 * 2) /* 0xFFFA Timer B 0 */
#define NMI_VECTOR          (14 * 2) /* 0xFFFC Non-maskable */
#define RESET_VECTOR        (15 * 2) /* 0xFFFE Reset [Highest Priority] */
```

Bild 1-14: Verfügbare Interrupt Vektoren der MSP430F44x-Familie

Bei einem Interrupt Vektor handelt es sich um einen Zeiger auf eine Interrupt Service Routine. Es gibt eine Vielzahl an Ereignissen, die wiederum eine Vielzahl verschiedener Interrupt Service Routinen aufrufen können, wobei jedem Ereignis eine bestimmte Interrupt Service Routine festzugeordnet ist.

Wird z.B. die Interrupt Freigabe für einen Eingangs Pin von Port 1 parametrisiert, dann wird bei einem Ereignis an diesem Pin (fallende oder steigende Flanke) ein bestimmtes Event-Flag gesetzt (P1IFG.x).

Wenn ein solches Event-Flag gesetzt wurde und die entsprechende Interrupt Freigabe ist aktiviert, dann wird automatisch die zugehörige ISR, in diesem Fall die intrinsic Funktion `_interrupt void Port1_in(void)` aufgerufen. Dazu muss dem Mikrocontroller die Adresse dieser Funktion im zugehörigen Interrupt-Vektor bekanntgegeben werden. Dies macht man durch, `#pragma vector=PORT1_VECTOR`. Diese Direktive schreibt man unmittelbar vor die ISR.

Beim Übersetzen wird dann die Adresse der ISR-Funktion in den angegebenen Vektor geschrieben. Ein Interrupt-Vektor wird in der 16-Bit-Adresse der entsprechenden Interrupt-Service-Routine programmiert.

Es wird empfohlen, für jeden Interrupt-Vektor eine zugeordnete Interrupt-Service-Routine bereitzustellen. Wird ein Interrupt für ein Ereignis freigegeben für das es keine ISR mit einem entsprechend aktualisierten Vektor gibt, dann führt der Controller einen Neustart nach Auftreten des Ereignisses aus.

Freigabe-Modul, Interrupt-Freigabe und Interrupt-Flags sind in dem SFR-Speicherbereich platziert. Die SFRs (Spezial Funktionen Register) befinden sich im unteren Adressbereich und sind als Byte-Format implementiert. Auf die SFRs wird unter Verwendung von Byte-Anweisungen zugegriffen.

Wirkungslinie der Interruptverarbeitung

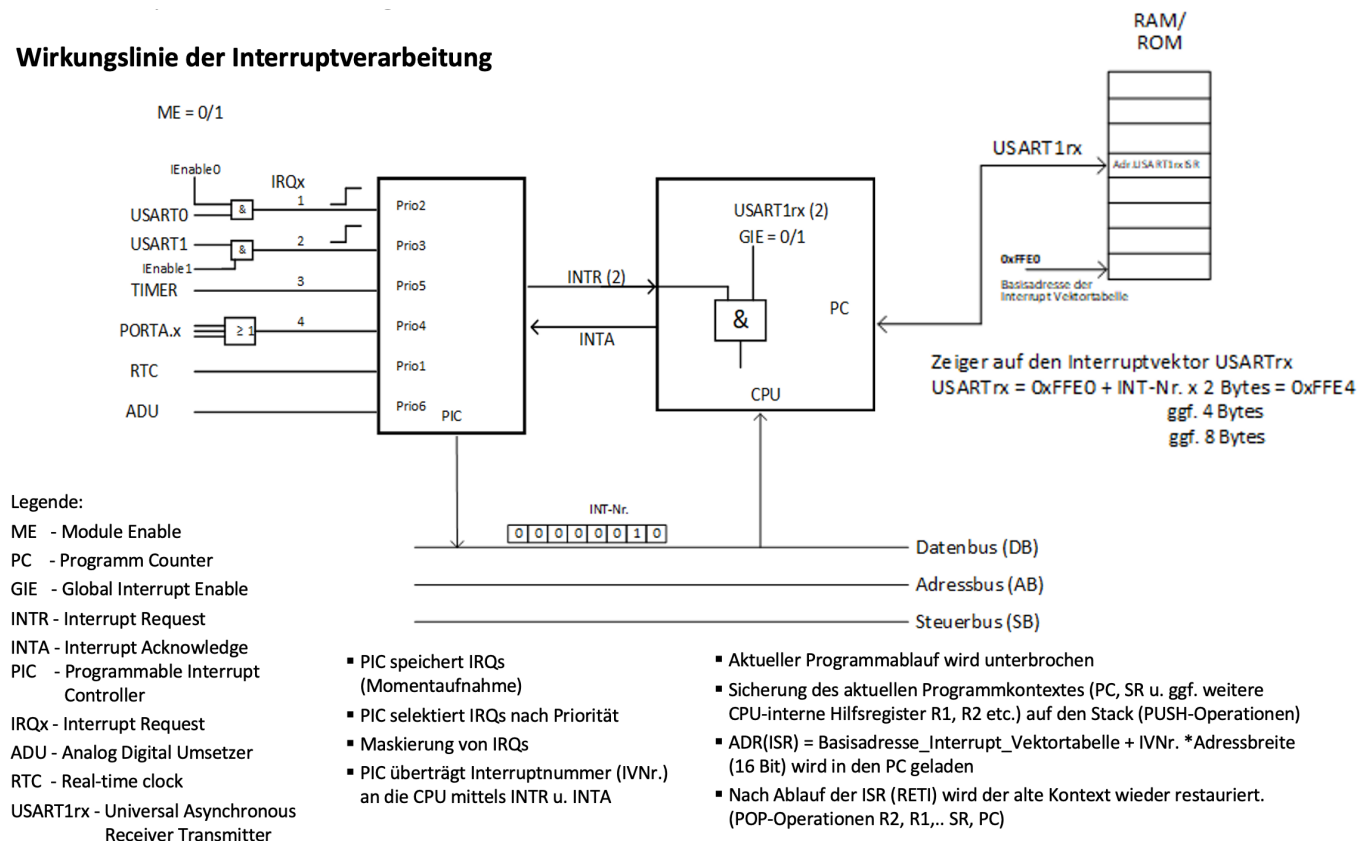


Bild 1-4: Wirkungslinie der Interruptverarbeitung

Beim Eintreffen eines Interrupts ist in der Interrupt Service Routine (ISR) das entsprechende, dem Interruptereignis zugeordnete Bit, zurückzusetzen.

Weiterführende Informationen enthält die PDF-Datei „MSP430_UserGuide_slau056l.pdf“.

1.5 Low Power Management des MSP430-449 STK2 Board

Der MSP420 kann in unterschiedlichen Betriebsarten betrieben werden. Interessant ist, dass die unterschiedlichen Betriebsarten des MSP430 einzig durch Manipulationen an den

Steuerbits des Basic-Clock-Moduls gebildet werden. Involviert sind vier im Statusregister angesiedelte Bits, die auch als Schalter bezeichnet werden können: SCG0, SCG1, OSCOFF und CPUOFF. Ihre Kombination ermöglicht die Taktabschaltung einzelner Controllersysteme, wodurch die so genannten Low-Power Modi gebildet werden.

Aus der Kombination der Bitzustände ergeben sich die folgenden fünf unterschiedlichen Betriebsarten:

Betriebszustand	SCG1	SCG0	OscOFF	CPUOFF	Stromaufnahme
Active Mode	0	0	0	0	200 μ A
LPM0	0	0	0	1	35 μ A
LPM1	0	1	0	1	
LPM2	1	0	0	1	
LPM3	1	1	0	1	0,8 μ A
LPM4	1	1	1	1	0,1 μ A

Bild 1-4: Stromaufnahme in Abhängigkeit des Betriebszustandes beim MSP430F449

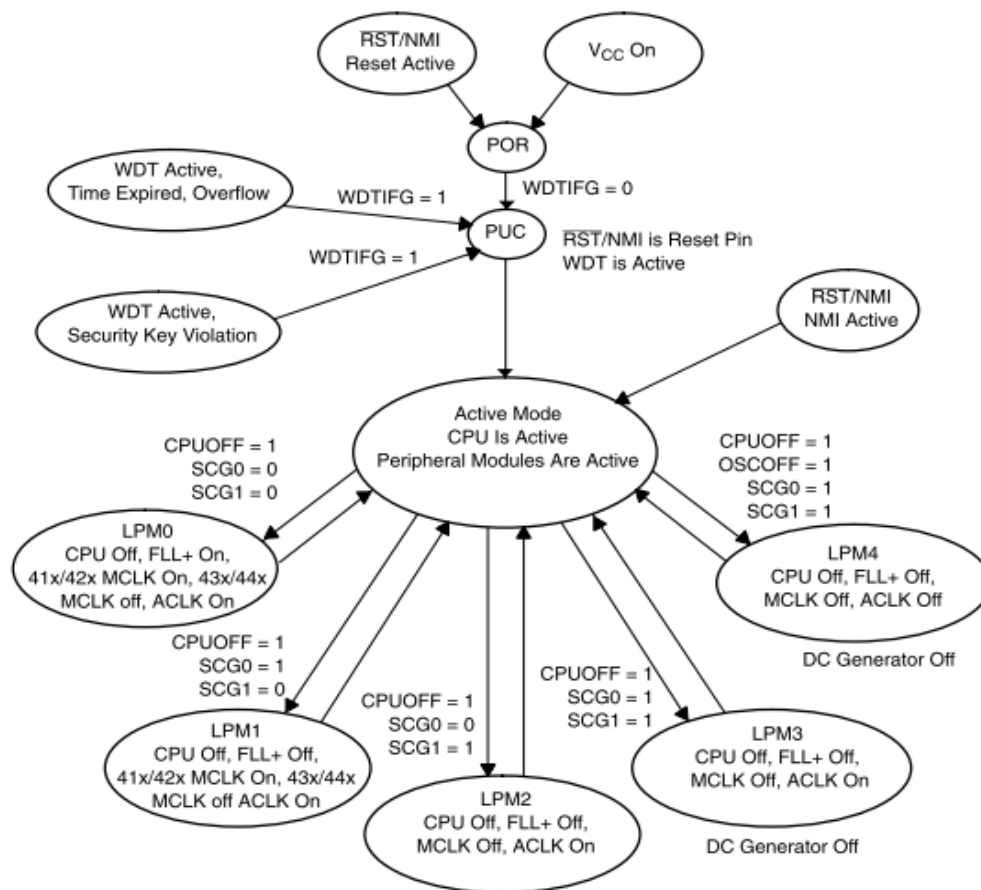


Bild 1-5: Low Power Mode Zustände beim der MSP430F44x Mikrocontrollerfamilie

SCG1	SCG0	OSCOFF	CPUOFF	Mode	CPU and Clocks Status
0	0	0	0	Active	CPU is active, all enabled clocks are active
0	0	0	1	LPM0	CPU, MCLK are disabled (41x/42x peripheral MCLK remains on) SMCLK, ACLK are active
0	1	0	1	LPM1	CPU, MCLK, DCO oscillator are disabled (41x/42x peripheral MCLK remains on) DC generator is disabled if the DCO is not used for MCLK or SMCLK in active mode SMCLK, ACLK are active
1	0	0	1	LPM2	CPU, MCLK, SMCLK, DCO oscillator are disabled DC generator remains enabled ACLK is active
1	1	0	1	LPM3	CPU, MCLK, SMCLK, DCO oscillator are disabled DC generator disabled ACLK is active
1	1	1	1	LPM4	CPU and all clocks disabled

Bild 1-6: Konfiguration der Low Power Modi im Prozessorstatusregister (PSW)

Ein auftretendes Interrupt Ereignis erweckt dabei immer den MSP430 aus allen Betriebsarten.

Weiterführende Informationen enthält die PDF-Datei „MSP430_UserGuide_slau056l.pdf“

2 TEILVERSUCH 0: INBETRIEBNAHME DES EMBEDDED SYSTEMS

2.1 Ziele und Aufgabenbeschreibung

Ziel des Teilversuchs „Versuch 0“ ist das Kennenlernen der Toolchain anhand eines einfachen vollständigen Beispielprogrammes, im Detail die

- Einarbeitung in die integrierte Entwicklungsumgebung IAR Embedded Workbench
- Kennenlernen des I/O-Systems des MSP430F449 Boards
- Einarbeitung in die Möglichkeiten des Embedded System STK2 der Firma Olimex.

Mit einem ersten Programm Versuch0.c soll die Programmerstellung für das Embedded System STK2 erlernt werden.

Eine auf dem Board vorhandene LED blinkt nach einem erfolgreichen Download und Start des Programms.

2.2 Versuchsvorbereitung und Testfragen

Machen Sie sich den Ablauf des Beispielprogramms „Versuch0.c“ klar. Informieren Sie sich über die im Programm verwendeten Bitoperationen. Machen Sie sich den Ablauf und die Auswirkungen auf die Ansteuerung der LED klar. Informieren Sie sich über die Grundlagen der I/O-Programmierung im Anhang.

2.3 Testfragen

Beantworten Sie folgende Fragen zum Programm

- Was bewirkt die Anweisung: `P1DIR = P1DIR | BIT3;` ?
- Was bewirkt die Anweisung: `P1OUT = P1OUT ^ 0x08;` ?

2.4 Programmbeispiel

Es soll die LED auf dem Entwicklungsboard blinken.

```
// Dateiname: Versuch0.c
// Dieses Programm dient dem grundsätzlichen Test der Toolchain und der
// Ueberpruefung der richtigen Einstellungen der IAR Entwicklungsumgebung
// Wenn alles i.O. blinkt eine LED auf dem Zielsystem
#include <msp430x44x.h>

volatile unsigned int i=5000; // if not volatile, the loop will be
                               // optimized away by the compiler

void main(void)
{
    WDTCTL = WDTPW + WDTHOLD; // Stop watchdog timer

    P1SEL = P1SEL & ~BIT3;    // Set P1.3 (LED) as digital I/O port
    P1DIR = P1DIR | BIT3;     // Set Pin P1.3 00001000b to output direction

    for (;;)                 // infinite loop
    {
        P1OUT = P1OUT ^ 0x08; // Toggle LED (=P1.3) using exclusive-OR

        while (i-- != 0);     // Delay
        i = 50000;
    }
} //end main()
```

Programmlisting Versuch0.c

2.5 Versuchsdurchführung und Anschluss des Boards

Der Anschluss des MSP430-449 STK2 Boards an den PC erfolgt programmierseitig über die USB-Schnittstelle des PCs und dem Olimex-Programmieradapter

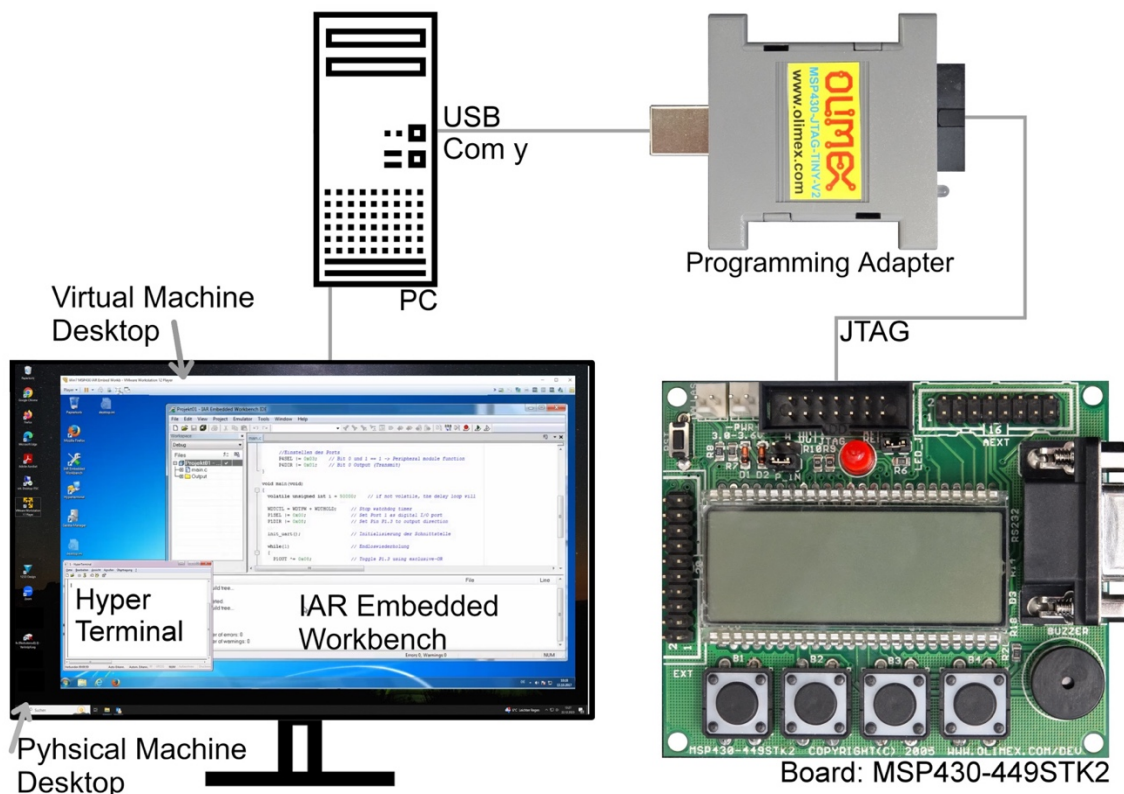


Bild 2-1: Anschluss des MSP430 Boards mit Spannungsversorgung über die JTAG-Schnittstelle

Wichtig:

Nach dem Starten der virtuellen Maschine müssen Sie sich innerhalb dem virtualisierten Windows mit dem Administrator Account anmelden, also

- a. Starten mit "Play virtual machine". Auf die evtl. erscheinende Nachfrage: "This Virtual Machine might have been moved or copied" -> "I moved it" antworten.
 - b. Am virtualisierten Windows anmelden mit "Administrator" als Benutzername (kein Passwort), nicht mit dem Studierenden-Account anmelden.
1. Schließen Sie das Entwicklungsboard über die USB-Kommunikationsschnittstelle des PCs an. Studieren Sie das Kapitel „Verwendete Hardware“ im Anhang.

2. Starten Sie die integrierte Entwicklungsumgebung (IDE) IAR Embedded Workbench auf dem PC. Lesen Sie hierzu als Vorbereitung die notwendigen Informationen im Kapitel „Handhabung der Embedded Toolchain.pdf“.

Legen Sie in der IAR Embedded Workbench eine neue Arbeitsumgebung (Workspace) an.

Legen Sie dann ein Projekt mit dem Namen „Versuch0“ an. Lesen Sie hierzu im Anhang das Kapitel IAR Embedded Workbench - Starten eines neuen Projekts in der IDE.

3. Übernehmen Sie das Beispielprogramm „Versuch0.c“ in das Projekt „Versuch0“. Bevor Sie das Programm kompilieren, prüfen Sie bitte nochmals die Einstellungen.
 1. Im Programmfenster der IAR Embedded Workbench IDE im Bereich des Workspace (linke Fensterhälfte): rechte Maustaste auf Versuch0 - Debug -> Options...
 2. Im Options Fenster:
 - Category: General Options -> Reiter Target -> Device -> MSP430F449
 - Category: Debugger -> Reiter Setup -> Driver -> FET Debugger
 - Category: FET Debugger -> Setup -> Connection -> Olimex USB

Achtung: Für jedes Projekt müssen erneut unter [Projekt] [Optionen] die Einstellungen für den Mikrokontrollertyp, den Debugger (Simulator oder FET Debugger) und das Debugger Interface (Olimex USB) eingestellt werden.

Wenn die Einstellungen in Ordnung sind, wird das Programm kompiliert und in das Zielsystem, Board MSP430-449STK2, geladen.

Starten Sie den Debugger durch Drücken des Buttons  in der Entwicklungsumgebung IAR Embedded Workbench und führen Sie einen Funktionstest.

4. Dokumentieren Sie den Funktionstest mit Bildern oder Video.

2.6 Abzuliefernde Ergebnisse

- Dokumentation der Versuchsvorbereitung mit Antworten auf die Testfragen.
- Video oder Bildstrecke zur Dokumentation des Funktionstests.
- Beantworten Sie folgende Frage:
 - Beschreiben Sie die verwendeten Register zur Initialisierung der I/O-Schnittstelle.

3 TEILVERSUCH 1: POLLING EINES TASTERS ZUR ANSTEUERUNG DER LED

Beeinflussung des Programmablaufs unter Verwendung der digitalen Input/Output-Portschnittstelle (GPIO-Unit³).

3.1 Ziele und Aufgabenbeschreibung

Beim nächsten Programm „Versuch1.c“ soll eine auf dem Board vorhandene LED kurz blinken, nachdem der Taster B5 betätigt wurde. Die LED quittiert so das Betätigen des Tasters.

Die Abfrage des Tasters B5 erfolgt durch zyklisches Einlesen des digitalen Pegels am Portpin P2.1, was als Polling bezeichnet wird.

Voraussetzung für diesen Versuch ist ein Verständnis für die Ein- und Ausgabe über die parallelen Ports P1 und Port 2 des MSP430F449 Mikrocontrollers, ebenso wie eine Strommessung für die Tabelle unter 5.2 soll durchgeführt werden kann.

3.2 Versuchsvorbereitung und Testfragen

Machen Sie sich den Ablauf des „Beispielprogramm Versuch1.c“ klar. Informieren Sie sich über die im Programm verwendeten Bitoperationen. Machen Sie sich den Ablauf und die Auswirkungen auf die Ansteuerung der LED klar. Informieren Sie sich über die Grundlagen der I/O-Programmierung im Anhang.

3.3 Testfragen

Beantworten Sie folgende Fragen zum nachstehenden Programm

- a) Erläutern Sie im Detail die folgenden Anweisung *while (P2IN & BIT1);* ?
- b) Würde die Anweisung *if (P2IN & BIT1)* statt *while (P2IN & BIT1);* ebenfalls funktionieren ?
- c) Was bewirkt die Anweisung *WDTCTL = WDTPW + WDTHOLD;* ?
- d) Was versteht man unter dem Begriff „Polling“ und inwieweit wird diese Vorgehensweise im vorliegenden Programmablauf verwendet?

3.4 Programmbeispiel

Das Testprogramm fragt zyklisch den Zustand des Portpins P2.1 ab und signalisiert bzw. quittiert den Zustand „1“ durch kurzes An- oder Ausschalten der LED.

```
// Dateiname: Versuch1.c
// Dieses Programm veranschlicht die Handhabung des General Parallel Input
// Output Moduls (GPIO)
// Wenn alles i.O., kann eine LED mittels eines Tasters gesteuert werden

#include <msp430x44x.h>

volatile unsigned int i = 500; // if not volatile, the loop will be
                               // optimized away by the compiler
```

³ für engl. General Purpose Input/Output Unit

```

void main(void)
{
    WDTCTL = WDTPW + WDTHOLD;           // Stop watchdog timer

    P1SEL = P1SEL & ~BIT3;              // Set P1.3 (LED) as digital I/O port
    P1DIR = P1DIR | BIT3;               // Set Pin P1.3 00001000b to output direction

    P2SEL = P1SEL & ~BIT1;              // Set P2.1 (Button B5) as digital I/O port
    P2DIR = P1DIR & ~BIT1;             // Set P2.1 00000001b to input direction

    while(1)                            // infinite loop
    {
        while (P2IN & BIT1);           // wait until button B5 is pressed

        P1OUT = P1OUT & ~0x08;         // LED on

        while (i-- != 0);              // Delay
        i = 500;

        P1OUT = P1OUT | 0x08;          // LED off

    } //end while
} //end main

```

Programmlisting Versuch1.c

3.5 Versuchsdurchführung und Anschluss des Boards

1. Schließen Sie das Entwicklungsboard über die USB-Kommunikationsschnittstelle des PCs an. Studieren Sie das Kapitel „Verwendete Hardware“ im Anhang.
2. Starten Sie die integrierte Entwicklungsumgebung (IDE) IAR Embedded Workbench auf dem PC. Lesen Sie hierzu als Vorbereitung die notwendigen Informationen im Kapitel „HandhabungderEmbeddedToolchain.pdf“.
3. Legen Sie in der IAR Embedded Workbench eine neue Arbeitsumgebung (Workspace) an.
 Legen Sie dann ein Projekt mit dem Namen „Versuch1“ an. Lesen Sie hierzu im Anhang das Kapitel *IAR Embedded Workbench - Starten eines neuen Projekts in der IDE*.
4. Übernehmen Sie das Beispielpogramm „Versuch1.c“ in das Projekt „Versuch1“. Bevor Sie das Programm kompilieren, prüfen Sie bitte nochmals die Einstellungen.
 1. Im Programmfenster der IAR Embedded Workbench IDE im Bereich des **Workspace** (linke Fensterhälfte): rechte Maustaste auf **Versuch1 – Debug -> Options...**
 2. Im **Options** Fenster:
 - **Category: General Options -> Reiter Target -> Device -> MSP430F449**
 - **Category: Debugger -> Reiter Setup -> Driver -> FET Debugger**
 - **Category: FET Debugger -> Setup -> Connection -> Olimex USB**

Achtung: Für jedes Projekt müssen erneut unter [Projekt] [Optionen] die Einstellungen für den Mikrocontrollertyp, den Debugger (Simulator oder FET- Debugger) und das Debugger Interface (Olimex USB) eingestellt werden.

Wenn die Einstellungen in Ordnung sind, wird das Programm kompiliert und in das Zielsystem, Board MSP430-449STK2, geladen.

Starten Sie den Debugger durch Drücken des Buttons  in der Entwicklungsumgebung IAR Embedded Workbench.

5. Testen Sie die Funktion des Programms „Versuch1.c“ im Zielsystem mit dem Hochsprachendebugger.
6. Dokumentieren Sie den Funktionstest mit Bildern oder Video.

3.6 Abzuliefernde Ergebnisse

- Dokumentation der Versuchsvorbereitung mit Antworten auf die Testfragen.
- Video oder Bildstrecke zur Dokumentation des Funktionstests.
- Beantworten Sie folgende Frage:

Beschreiben Sie über welches Eingabesystem die Beeinflussung des Programms realisiert ist und welche anderen Möglichkeiten des MSP430-449 STK2 Boards hierfür zur Verfügung stehen.

4 TEILVERSUCH 2: INTERRUPT-BETRIEB

Interrupt-Verarbeitung des Mikrocontroller MSP430F449.

4.1 Ziele und Aufgabenbeschreibung

Ziel des Versuches ist das Kennenlernen der Interrupt-Verarbeitung des Mikrocontrollers MSP430F449.

Im Teilversuch 2 soll nun auf das energieintensive Polling verzichtet werden und eine ereignisgesteuerte, d. h. interruptorientierte Implementierung des Anwendungsfalles angestrebt werden.

Wird der Taster B5 betätigt, so wird intern ein Interrupt (hier Portinterrupt) ausgelöst und die zugeordnete Interruptservicefunktion `ISR_Port2()` gestartet. In der `ISR_Port2()` wird die LED wechselweise ein- bzw. ausgeschaltet.

4.2 Versuchsvorbereitung und Testfragen

Machen Sie sich den Ablauf des „Beispielprogramm Versuch2.c“ klar. Informieren Sie sich über die im Programm verwendeten Bitoperationen und die verwendete Interruptfunktionalität des Mikrocontrollers MSP430F449.

Machen Sie sich den Ablauf und die Auswirkungen auf die Ansteuerung der LED klar. Informieren Sie sich über die Grundlagen der Interrupt-Programmierung im Anhang.

Lesen Sie im Anhang das Kapitel 6.3 Interrupt-Logik des MSP430-449 STK2 Board.

4.3 Testfragen

Beantworten Sie folgende Fragen zum Programm

- a) Was bewirkt die Anweisung `P2IES = P2IES | BIT1;` ?
- b) Was bewirkt die Anweisung `_EINT();` ?
- c) Wie würde sich das Programm verhalten, wenn die Anweisung `P2IFG = P2IFG & ~BIT1;` in der Interrupt Service Routine `ISR_Port2(void)` entfernt wird?

4.4 Programmbeispiel

Das Testprogramm initialisiert einen Interrupthandler (ISR) für Port2 und läuft danach in einer Endlosschleife.

Wird der Interrupt ausgelöst, wird das Hauptprogramm unterbrochen und vor Weiterführung im Interrupt-Programm der Auslöser untersucht. Ist Bit1 als Auslöser ermittelt, wird im Interrupt-Programm die LED am Port1-Bit3 an- oder ausgeschaltet und anschließend das Interruptflag gelöscht.

```
// Dateiname: Versuch2.c
// Dieses Programm dient dem Veranschaulichen der Interrupt-Funktionalität
// Wenn alles i.O. kann eine LED über eine Taster-Interrupt beeinflusst werden
```

```

#include <msp430x44x.h>

volatile unsigned int i=5000;           // if not volatile, the loop will be
                                         // optimized away by the compiler

void main(void)
{
    WDTCTL = WDTPW + WDTHOLD;           // Stop watchdog timer

    P1SEL = P1SEL & ~BIT3;               // Set P1.3 (LED) as digital I/O port
    P1DIR = P1DIR | BIT3;               // Set Pin P1.3 00001000b to output direction

    P2SEL = P2SEL & ~BIT1;               // Set P2.1 (Button B5) as digital I/O port
    P2DIR = P2DIR & ~BIT1;               // Set P2.1 00000001b to input direction

    P2IE = P2IE | BIT1;                 // Activate interrupt for P2.1
    P2IES = P2IES | BIT1;               // Detect high-low transition

    _EINT();                             // CPU general interrupt enable

    while(1);                           // Endlosschleife

} //end main

#pragma vector = PORT2_VECTOR
__interrupt void ISR_Port2(void)
{
    P1OUT = P1OUT ^ 0x08;                // LED-Toggle P1.3 using exclusive-OR
    P2IFG = P2IFG & ~BIT1;              // Clears pending interrupt flag
} //end interrupt ISR

```

Programmlisting Versuch2.c

4.5 Versuchsdurchführung und Anschluss des Boards

1. Schließen Sie das Entwicklungsboard über die USB-Kommunikationsschnittstelle des PCs an.
2. Starten Sie die integrierte Entwicklungsumgebung (IDE) IAR Embedded Workbench auf dem PC.
Legen Sie in der IAR Embedded Workbench eine neue Arbeitsumgebung (Workspace) an.
Legen Sie dann ein Projekt mit dem Namen „Versuch2“ an. Lesen Sie hierzu im Anhang das Kapitel *IAR Embedded Workbench - Starten eines neuen Projekts in der IDE*.
3. Übernehmen Sie das Beispielprogramm „Versuch2.c“ in das Projekt „Versuch2“. Bevor Sie das Programm kompilieren, prüfen Sie bitte nochmals die Einstellungen.
 1. Im Programmfenster der IAR Embedded Workbench IDE im Bereich des **Workspace** (linke Fensterhälfte): rechte Maustaste auf **Versuch2 – Debug -> Options...**
 2. Im **Options** Fenster:

- **Category: General Options -> Reiter Target -> Device -> MSP430F449**
- **Category: Debugger -> Reiter Setup -> Driver -> FET Debugger**
- **Category: FET Debugger -> Setup -> Connection -> Olimex USB**

Achtung: Für jedes Projekt müssen erneut unter [Projekt] [Optionen] die Einstellungen für den Mikrokontrollertyp, den Debugger (Simulator oder FET Debugger) und das Debugger Interface (Olimex USB) eingestellt werden.

Wenn die Einstellungen in Ordnung sind, wird das Programm kompiliert und in das Zielsystem, Board MSP430-449STK2, geladen.

Starten Sie den Debugger durch Drücken des Buttons  in der Entwicklungsumgebung IAR Embedded Workbench.

4. Testen Sie die Funktion des Programms „Versuch2.c“ im Zielsystem mit dem Hochsprachendebugger.
5. Messen Sie die Stromaufnahme und tragen Sie diese in die Tabelle „Stromaufnahme des MSP430-449 STK2 Boards“ in Kap. 5.2. ein
6. Dokumentieren Sie den Funktionstest mit Bildern oder Video.
7. Manchmal prellen die Tasten B5 und B6, so dass derselbe Interrupt mehrmals kurz hintereinander ausgelöst wird. Die LED geht dann schnell an und wieder aus. Die Lösung hierzu wäre eine Entprellverzögerung. Diese sorgt dafür, dass die Interrupt-Service-Routine (ISR_Port2) nur einmal pro Benutzereingabe ausgelöst wird.

Erweitern Sie das Programm entsprechend der folgenden Anforderung:

Nach dem Eintreffen eines Interrupts soll eine bestimmte Zeit gewartet werden, um das Hüpfen des Buttons zu überbrücken (Debounce delay).

```
for (int i=10000; i>0; i--);      // Entprellverzögerung
```

8. Testen Sie das erweiterte Programm und prüfen Sie, ob das Pellen durch die kurze Verzögerungszeit beseitigt wird. Ändern Sie ggf. die Verzögerungszeit. Bitte demonstrieren Sie die Funktion abschließend dem Betreuer.

4.6 Abzuliefernde Ergebnisse

- Dokumentation der Versuchsvorbereitung mit Antworten auf die Testfragen.
- Video oder Bildstrecke zur Dokumentation des Funktionstests.
- Beantworten Sie folgende Frage:
 - a) Wie hat sich die Verwendung eines Interrupts auf den Ablauf des Hauptprogramms ausgewirkt?
 - b) Warum toggelt die LED manchmal nicht, obwohl der Taster B5 betätigt wurde?

5 TEILVERSUCH 3: ENERGIEMANAGEMENT DES MSP430F449 NUTZEN

5.1 Ziele und Aufgabenbeschreibung

Das hier gezeigte Programm soll die im Vorversuch aufgezeigte Funktionalität unter Nutzung der Energiesparmöglichkeiten des MSP430-449 Mikrocontrollers realisieren.

- Einarbeitung in die Energiesparmöglichkeiten der Mikrocontroller-Familie MSP430F499, vgl. Low Power Modes LPM0 - LPM4.
- Messung der Stromaufnahme im Interrupt-Betrieb.

Um die Leistungsaufnahme der Embedded Hardware zu kontrollieren, wird die Stromaufnahme des Embedded Boards MSP430F449-STK2 erfasst und protokolliert.

Wichtig: Damit die Messung der Stromaufnahme der Embedded Hardware nicht durch den JTAG-Programmieradapter verfälscht wird, muss dieser JTAG-Programmieradapter durch Ziehen des entsprechenden Jumpers P_IN von dem Embedded Board MSP430F449-STK2 getrennt werden. Außerdem muss die Spannungsversorgung der Embedded Hardware ausschließlich mittels Batterie-Pack erfolgen.

5.2 Versuchsvorbereitung und Testfragen

Machen Sie sich den Ablauf des „Beispielprogramm Versuch3.c“ klar. Informieren Sie sich über die im Programm verwendeten Energieeinsparmöglichkeiten des MSP430-449 Controllers.

Lesen Sie im Anhang das Kapitel 7.1 Strommessung am MSP430-449 STK2 Board und das Kapitel 1.5 Low Power Management (LPM) des MSP430-449 STK2 Board.

Die Stromaufnahme soll im Betrieb einmal mit und einmal ohne Energiesparmodus gemessen und in der nachfolgenden Tabelle protokolliert werden.

Zur Protokollierung der gemessenen Stromwerte dient folgende Tabelle:

Fall	Betriebsart	I [μ A]
1	Programm im Polling-Betrieb , mit CPU Active mode , LED Ein (Versuch1.c)	
2	Programm im Polling-Betrieb , mit CPU Active mode , LED Aus (Versuch1.c)	
3	Programm im Interrupt-Betrieb , mit CPU Active mode , LED Ein (Versuch2.c)	
4	Programm im Interrupt-Betrieb , mit CPU Active mode , LED Aus (Versuch2.c)	
5	Programm im Interrupt-Betrieb , mit Low Power Mode (LPM2) , LED Ein (Versuch3.c)	
6	Programm im Interrupt-Betrieb , mit Low Power Mode (LPM2) , LED Aus (Versuch3.c)	
7	Programm im Interrupt-Betrieb , mit Low Power Mode (LPM4) , LED Aus (Versuch3.c)	

Tabelle 1: Stromaufnahme des MSP430-449 STK2 Boards

5.2.1 Testfragen

Beantworten Sie folgende Fragen zum Programm

- Was bewirkt die Anweisung `BIS_SR(LPM4_bits);` ?
- Was bewirkt hingegen die Anweisung `BIC_SR_IRQ(LPM4_bits);` ?
- Wie verändert sich das Verhalten des Programmes, wenn die Anweisung `BIC_SR_IRQ(LPM4_bits);` durch die Anweisung `BIS_SR(LPM4_bits);` ersetzt würde?

5.3 Programmbeispiel und Funktionsweise

Das nachstehende Programmbeispiel installiert eine Interrupt-Service-Routine (ISR) für den Port2. Innerhalb einer Endlosschleife wechselt der Mikrocontroller zunächst in den Energiesparmodus „Low Power Modus 2“ (LPM2) .

Durch Betätigen des schwarzen Tasters B5 wird der Port2-Interrupt ausgelöst. Dadurch wechselt der Mikrocontroller wieder in den aktiven Modus und verbleibt, bedingt durch die Anweisung `_BIC_SR_IRQ(LPM4_bits)`, weiterhin in diesem Modus. Auf diese Weise gelangt die CPU wieder in das Hauptprogramm `main()` zurück und kann nun weitere Anweisungen ausführen.

Da es sich bei dem `PORT2_VECTOR` um einen Sammelinterrupt handelt, muss zunächst mittels der Anweisung `if (P2IFG & BIT1)` geprüft werden, ob der Interrupt tatsächlich durch den Taster B5 (schwarz) ausgelöst wurde.

Wird der Taster B5 als Interruptereignis identifiziert, blinkt die LED einmalig; danach wird das Interruptereignis (=Interruptflag) durch die Anweisung `P2IFG=P2IFG & ~BIT1;` wieder gelöscht und der Mikrocontroller erneut mittels der Anweisung `_BIS_SR(LPM4_bits);` in den Schlafmodus versetzt.

```
// Dateiname: Versuch3.c
// Funktion des Programms:
// Das Betaetigen des schwarzen Tasters B5 wird durch ein kurzes
// LED-Blinken quittiert

#include <msp430x44x.h>

volatile unsigned int i = 30000; // if not volatile, the delay loop will
char state;

void main(void)
{
    WDTCTL = WDTPW + WDTHOLD; // Stop watchdog timer

    P1SEL = P1SEL & ~BIT3; // Set P1.3 (LED) as digital I/O port
    P1DIR = P1DIR | BIT3; // Set Pin P1.3 00001000b to output direction
    P1OUT = P1OUT | 0x08; // LED off

    P2SEL = P2SEL & ~BIT1; // Set P2.1 (Button B5) as digital I/O port
    P2DIR = P2DIR & ~BIT1; // Set P2.1 00000001b to input direction

    P2IE = P2IE | BIT1; // Activate interrupt for Portpin P2.1
    P2IES = P2IES | BIT1; // Detect high-low transition

    _EINT(); // CPU general interrupt enable

    while(1) // infinite loop
    {
        _BIS_SR(LPM4_bits); // enter LPM4, program stops

        if (state ==1) // check B5 (P2.1) for interrupt event
        {
            P1OUT = P1OUT & ~0x08; // LED on

            while (i-- != 0); // Delay
            i = 30000;

            P1OUT = P1OUT | 0x08; // LED off

            state = 0;
        } //end if

    } //end while

} //end main

#pragma vector = PORT2_VECTOR
__interrupt void ISR_Port2(void)
{
    _BIC_SR_IRQ(LPM4_bits); // modify Prozessor Status Word on the Stack

    if (P2IFG & BIT1)
        state = 1;
}
```

```
P2IFG = P2IFG & ~BIT1;      // clear interrupt flag

} //return from Interrupt (RETI)
```

Programmlisting Versuch3.c

5.4 Versuchsdurchführung und Anschluss des Boards

1. Schließen Sie das Entwicklungsboard über die USB-Kommunikationsschnittstelle des PCs an. Studieren Sie das Kapitel „Verwendete Hardware“ im Anhang.
2. Starten Sie die integrierte Entwicklungsumgebung (IDE) IAR Embedded Workbench auf dem PC. Lesen Sie hierzu als Vorbereitung die notwendigen Informationen im Kapitel „HandhabungderEmbeddedToolchain.pdf“.

Legen Sie in der IAR Embedded Workbench eine neue Arbeitsumgebung (Workspace) an.

Legen Sie dann ein Projekt mit dem Namen „Versuch3“ an. Lesen Sie hierzu im Anhang das Kapitel *IAR Embedded Workbench - Starten eines neuen Projekts in der IDE*.

3. Übernehmen Sie das Beispielprogramm „Versuch3.c“ in das Projekt „Versuch3“. Bevor Sie das Programm kompilieren, prüfen Sie bitte nochmals die Einstellungen.
 1. Im Programmfenster der IAR Embedded Workbench IDE im Bereich des **Workspace** (linke Fensterhälfte): rechte Maustaste auf **Versuch3 – Debug -> Options...**
 2. Im **Options** Fenster:
 - **Category: General Options -> Reiter Target -> Device -> MSP430F449**
 - **Category: Debugger -> Reiter Setup -> Driver -> FET Debugger**
 - **Category: FET Debugger -> Setup -> Connection -> Olimex USB**

Achtung: Für jedes Projekt müssen erneut unter [Projekt] [Optionen] die Einstellungen für den Mikrokontrollertyp, den Debugger (Simulator oder FET Debugger) und das Debugger Interface (Olimex USB) eingestellt werden.

Wenn die Einstellungen in Ordnung sind, wird das Programm kompiliert und in das Zielsystem, Board MSP430-449STK2, geladen.

Starten Sie den Debugger durch Drücken des Buttons  in der Entwicklungsumgebung IAR Embedded Workbench.

4. Testen Sie die Funktion des Programms „Versuch3.c“ im Zielsystem mit dem Hochsprachendebugger.
5. Messen Sie den Strom des laufenden Programms und protokollieren Sie diesen in der Tabelle „Stromaufnahme des MSP430-449 STK2 Boards“ in Kapitel 5.2.. Die Spannungsversorgung der Embedded-Hardware soll mittels Batterie-Pack

erfolgen, damit während der Messung der Stromaufnahme die Verbindung zum PC getrennt werden kann.

6. Dokumentieren Sie den Funktionstest mit Bildern oder Video.

5.5 Abzuliefernde Ergebnisse

- Dokumentation der Versuchsvorbereitung mit Antworten auf die Testfragen.
- Dokumentieren Sie den Funktionstest mit Bildern oder Video.
- Warum blinkt die LED nach Betätigen des schwarzen Tasters manchmal zweimal?
- Wie interpretieren Sie die gemessenen Werte in der Tabelle „Stromaufnahme des MSP430-449 STK2 Boards“ in Kap. 5.2 ?

6 TEILVERSUCH 4: TRANSFERAUFGABE

6.1 Ziele und Aufgabenbeschreibung

Im Versuch „Versuch 4“ soll die Funktionalität des Programms „*Versuch3.c*“ aus Kap. 5.3 wie folgt erweitert werden:

- a) Durch das Betätigen des schwarzen Tasters B5 soll die LED **eingeschaltet** werden,
- b) durch das Betätigen des roten Tasters B6 soll die LED **ausgeschaltet** werden.

Für eine möglichst energiesparende Implementierung sollen die Tastenereignisse mit Hilfe von Interrupts ausgewertet werden.

Ebenso sollen die Low Power Modi des Mikrocontrollers genutzt werden, um eine energiesparende Betriebsweise zu realisieren.

Die Steuerung der LED soll im Hauptprogramm *main()* realisiert werden.

6.2 Versuchsdurchführung und Anschluss des Boards

- 7. Schließen Sie das Entwicklungsboard über die USB-Kommunikationsschnittstelle des PCs an. Studieren Sie das Kapitel „Verwendete Hardware“ im Anhang.

- 8. Starten Sie die integrierte Entwicklungsumgebung (IDE) IAR Embedded Workbench auf dem PC. Lesen Sie hierzu als Vorbereitung die notwendigen Informationen im Kapitel „Handhabung der Embedded Toolchain.pdf“.

Legen Sie in der IAR Embedded Workbench eine neue Arbeitsumgebung (Workspace) an.

Legen Sie dann ein Projekt mit dem Namen „*Versuch4*“ an. Lesen Sie hierzu im Anhang das Kapitel *IAR Embedded Workbench - Starten eines neuen Projekts in der IDE*.

- 9. Übernehmen Sie das Beispielprogramm „*Versuch3.c*“ in das Projekt „*Versuch4*“. Bevor Sie das Programm kompilieren, prüfen Sie bitte nochmals die Projekteinstellungen.
- 3. Im Programmfenster der IAR Embedded Workbench IDE im Bereich des **Workspace** (linke Fensterhälfte): rechte Maustaste auf **Versuch4 – Debug -> Options...**
- 4. Im **Options** Fenster:
 - **Category: General Options -> Reiter Target -> Device -> MSP430F449**
 - **Category: Debugger -> Reiter Setup -> Driver -> FET Debugger**
 - **Category: FET Debugger -> Setup -> Connection -> Olimex USB**

Achtung: Für jedes Projekt müssen erneut unter [Projekt] [Optionen] die Einstellungen für den Mikrokontrollertyp, den Debugger (Simulator oder FET Debugger) und das Debugger Interface (Olimex USB) eingestellt werden.

Wenn die Einstellungen in Ordnung sind, wird das Programm kompiliert und in das Zielsystem, Board MSP430-449STK2, geladen.

Starten Sie den Debugger durch Drücken des Buttons  in der Entwicklungsumgebung IAR Embedded Workbench.

10. Erweitern Sie das Programm .c“ im Zielsystem mit dem Hochsprachendebugger.

11. Erweitern Sie das Programm gemäss der folgenden Anforderungen:

- a) Durch das Betätigen des schwarzen Tasters B5 soll die LED eingeschaltet werden,**
- b) durch das Betätigen des roten Tasters B6 soll die LED ausgeschaltet werden.**

12. Testen Sie das Programm und Dokumentieren Sie den Funktionstest anhand von Bildern oder Videos.

6.3 Abzuliefernde Ergebnisse

- Dokumentation der Versuchsvorbereitung mit Antworten auf die Testfragen.
- **Komplettes Listing des erweiterten Programms.**
- Video oder Bilder zur Dokumentation des Funktionstests.

6.4 Programmbeispiel und Lösung

```
// Dateiname: Versuch4.c
// Funktion des Programms:
// a) Der schwarze Taster B5 schaltet die LED ein.
// b) Der rote Taster B6 schaltet die LED aus.
// Das Programm nutzt die Energiesparmodus LPM4

#include <msp430x44x.h>
char state;

void main(void)
{
    WDTCTL = WDTPW + WDTHOLD;           // Stop watchdog timer

    P1SEL = P1SEL & ~BIT3;              // Set P1.3 (LED) as digital I/O port
    P1DIR = P1DIR | BIT3;                // Set Pin P1.3 00001000b to output direction
    P1OUT = P1OUT | 0x08;                // LED off

    P2SEL = P2SEL & ~BIT1;              // Set P2.1 (Button B5) as digital I/O port
    P2DIR = P2DIR & ~BIT1;              // Set P2.1 00000001b to input direction
    P2IE  = P2IE  | BIT1;               // Activate interrupt for Portpin P2.1
    P2IES = P2IES | BIT1;               // Detect high-low transition

    // Set P2.2 (Button B6) as digital I/O port
    // Set P2.2 00000010b to input direction
    // Activate interrupt for Portpin P2.2
    // Detect high-low transition

    _EINT();                             // CPU general interrupt enable

    while(1)                             // infinite loop
    {
        _BIS_SR(LPM4_bits);             // enter LPM4, program stops
    }
}
```

```

switch (state)
{
    case 1: P1OUT = P1OUT & ~0x08;      // LED on
           break;

                                           // LED off

    default: break;

} //end switch
} //end while
} //end main

#pragma vector = PORT2_VECTOR
__interrupt void ISR_Port2(void)
{
    _BIC_SR_IRQ(LPM4_bits);    // modify Prozessor Status Word on the Stack

    if (P2IFG & BIT1)          // check black B5 (P2.1) for interrupt event
    {
        state = 1;             // switch LED on in routine main()
        P2IFG = P2IFG & ~BIT1; // clear interrupt flag BIT1
    }

                                // check red B6 (P2.2) for interrupt event

                                // switch LED off in routine main()
                                // clear interrupt flag BIT2

} //return from Interrupt (RETI)

```

Programmlisting Versuch4.c

6.5 Abzuliefernde Ergebnisse

- Dokumentation der Versuchsvorbereitung mit Antworten zu die Testfragen.
- Dokumentieren Sie den Funktionstest mit Bildern oder Videos.
- Programmlisting des erweiterten Programms

7 ANHANG

7.1 Strommessung am MSP430-449 STK2 Board

Wird das Batterie-Pack zur Stromversorgung angeschlossen, muss die Spannungsversorgung über die JTAG-Schnittstelle unterbrochen werden. Hierzu ist der Jumper P_IN zu entfernen. Ebenso ist der JTAG-Programmieradapter selbst zu entfernen, damit keine Stromversorgung durch den JTAG-Programmieradapter erfolgt.

Anschließend ist zur Spannungsversorgung das Batterie-Pack aufzustecken, siehe Bild „Nachher“.

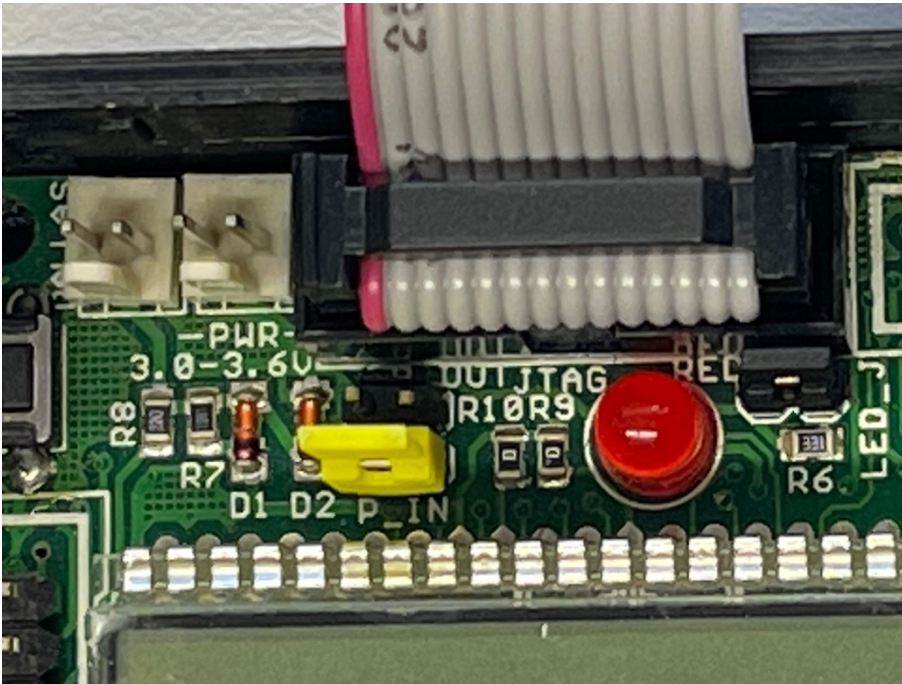


Bild 7-1: Vorher

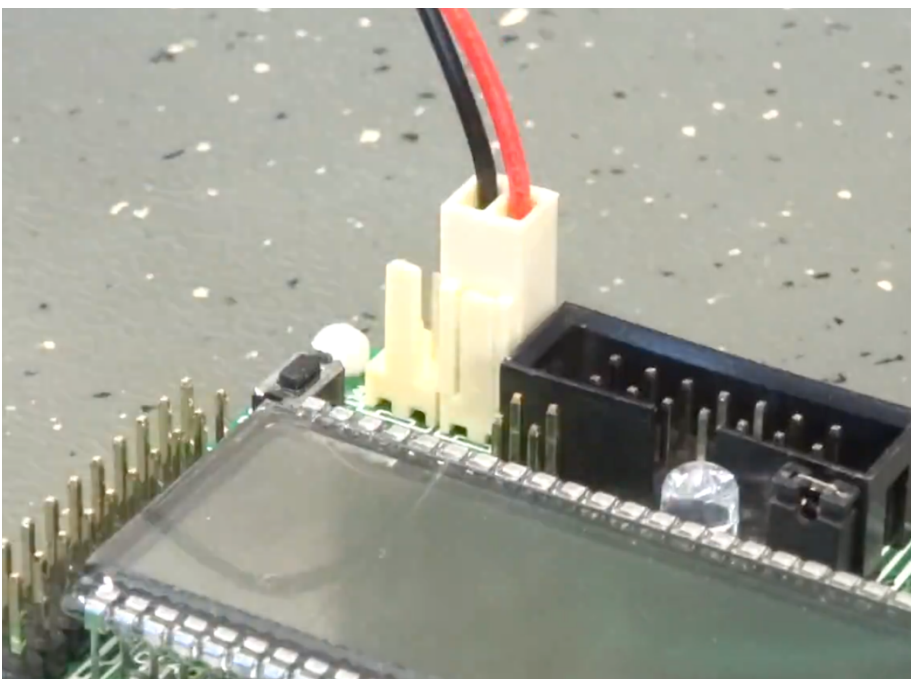


Bild 7-2: Nachher

Siehe hierzu auch das Video „EmbeddedSystemsVideoanleitungV1.3.mp4“, Strommessung ab 6:10min

7.2 Verwendete Hardware

Im Praktikum wird der Prozessor MSP430F449 von Texas Instruments verwendet.

Der Prozessor bietet folgende Möglichkeiten:

- Programmspeicherung im Flash
- Maximale Programmgröße 60kB
- 2048 Byte RAM
- 48 I/O Pins
- Ansteuerung von bis zu 160 LCD Segmenten
- Integrierter Analog Komparator
- 5 integrierte Hardwarezähler (Timer)
- Universal Asynchronous Receiver Transmitter (UART) hardwareseitig vorhanden

Das Datenblatt des Prozessors ist im Internet bei Texas Instruments zu erhalten, siehe hierzu das Dokument „MSP430_Datasheet_msp430f449.pdf“.

<http://focus.ti.com/docs/prod/folders/print/msp430f449.html>

Das verwendete Entwicklungsboard MSP430F449-STK2 der Fa. Olimex stellt folgende Optionen zur Verfügung:

- JTAG Programmier und Debug Connector
- RST/NMI pin pull-up
- 32 768 Hz crystal oscillator
- 8 Mhz high frequency oscillator (works only with 3.6V power supply)
- Ein RS232 Interface
- Dallas touch button port
- Frequenzeingang
- LCD Display
- 4 Taster
- 1 Buzzer
- 4 digital, 1 analog extension ports

7.3 Handhabung der Embedded Toolchain

7.3.1 Virtuelle Maschine

1. Am Rechner als Benutzer „Student“ mit dem Passwort „stuDent“ anmelden
2. **VMware Workstation Player** (nicht VMWare Workstation Pro) starten
3. **Open Virtual Machine** ausführen
4. Unter Laufwerk **C:** das Verzeichnis **VMWare Images \ Win7 MSP430 IAR Embed Workbench** und die darin liegende Datei **Windows 7** öffnen.
5. Die virtuelle Maschine starten mit **Play virtual machine**. Auf die evtl. erscheinende Nachfrage: **“This Virtual Machine might have been moved or copied”** -> **“I moved it”** antworten. Am virtualisierten Windows anmelden mit “Administrator” als Benutzername (kein Passwort), nicht mit dem Studierenden-Account anmelden.

Das Desktop-Fenster der virtuellen Maschine sollte auf Vollbild vergrößert werden. Über die Leiste am oberen Bildrand des Desktops können z.B. USB-Sticks an der virtuellen Maschine angemeldet werden. Dazu bitte wie folgt vorgehen: Auswahlpunkt **Virtual Machine -> Removable Devices -> „Name des USB-Sticks“ -> Connect (Disconnect from host)**

7.3.2 IAR Embedded Workbench - Starten eines neuen Projekts in der IDE

5. **IAR Embedded Workbench** starten.
6. Einen neuen Workspace anlegen mit: **File -> New -> Workspace**
7. Ein neues Projekt erstellen mit: **Project -> Create New Project... -> C -> main -> OK**
8. Im **Speichern unter** Dialog unter **Eigenen Dateien** einen neuen Ordner (z. B. Name **V1_1**) anlegen und das Projekt in diesem Ordner mit dem Namen **V1_1** speichern.
9. Im Programmfenster der IAR Embedded Workbench IDE im Bereich des **Workspace** (linke Fensterhälfte): rechte Maustaste auf **V1_1 – Debug -> Options...**
10. Im **Options** Fenster:
 - **Category: General Options -> Reiter Target -> Device -> MSP430F449**
 - **Category: Debugger -> Reiter Setup -> Driver -> FET Debugger**
 - **Category: FET Debugger -> Setup -> Connection -> Olimex USB**
11. Der Programm-Code kann mittels Cut & Paste direkt aus dem PDF-Dokument (vorher aus eLearning herunterladen) entnommen werden. Hierzu den Programmcode im PDF-Dokument markieren, kopieren und in der IAR Embedded Workbench im Unterfenster **main.c** einfügen und dabei den Code-Vorschlag der Workbench überschreiben.
12. Nun kann der Programmcode entsprechend der Aufgabenstellung bearbeitet werden.

7.3.3 Kompilieren und Downloaden des Programmes in das Zielsystem

1. In der drop-down Liste unter **Project -> Rebuild All -> Save Workspace As -> V1_1 -> Speichern** wird der aktuelle Workspace gespeichert.
2. Wenn im Unterfenster **Messages** keine **Errors** oder **Warnings** aufgezzeigt werden:
3. **Project -> Download and Debug**
4. Das Programm ist nun zur Ausführung auf der Zielhardware bereit:
 - Unter dem Menüpunkt **Debug** stehen verschiedene Möglichkeiten des Debuggens bereit. Mit **Go** bzw. **F5** kann man das Programm kontinuierlich laufen lassen.

Hinweis: Falls das Programm IAR Embedded Workbench neu gestartet werden sollte, sind alle Einstellungen erneut vorzunehmen.

7.4 Feedbackformular zum Versuch

Die Dokumentation in der Anlage ist verständlich.

trifft voll zu trifft überhaupt nicht zu

o-----o-----o-----o-----o-----o-----o

Teilversuch 1: Polling eines Tasters zur Ansteuerung der LED

Die Versuchsbeschreibung ist verständlich.

trifft voll zu trifft überhaupt nicht zu

o-----o-----o-----o-----o-----o-----o

Der Versuch konnte in der vorgegebenen Zeit durchgeführt werden.

o-----o-----o-----o-----o-----o-----o

Die Lernziele wurden erreicht

o-----o-----o-----o-----o-----o-----o

Verbesserungsvorschläge:

Teilversuch 2: "Interrupt-Betrieb"

Die Versuchsbeschreibung ist verständlich.

trifft voll zu trifft überhaupt nicht zu

o-----o-----o-----o-----o-----o-----o

Der Versuch konnte in der vorgegebenen Zeit durchgeführt werden.

o-----o-----o-----o-----o-----o-----o

Die Lernziele wurden erreicht

o-----o-----o-----o-----o-----o-----o

Verbesserungsvorschläge:

Teilversuch 3: Energiemanagement des MSP430-449 STK2 Boards

Die Versuchsbeschreibung ist verständlich.

trifft voll zu trifft überhaupt nicht zu

o-----o-----o-----o-----o-----o-----o

Der Versuch konnte in der vorgegebenen Zeit durchgeführt werden.

o-----o-----o-----o-----o-----o-----o

Die Lernziele wurden erreicht

o-----o-----o-----o-----o-----o-----o

Verbesserungsvorschläge:
